رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

Data Preparation for Exploration

Session 5

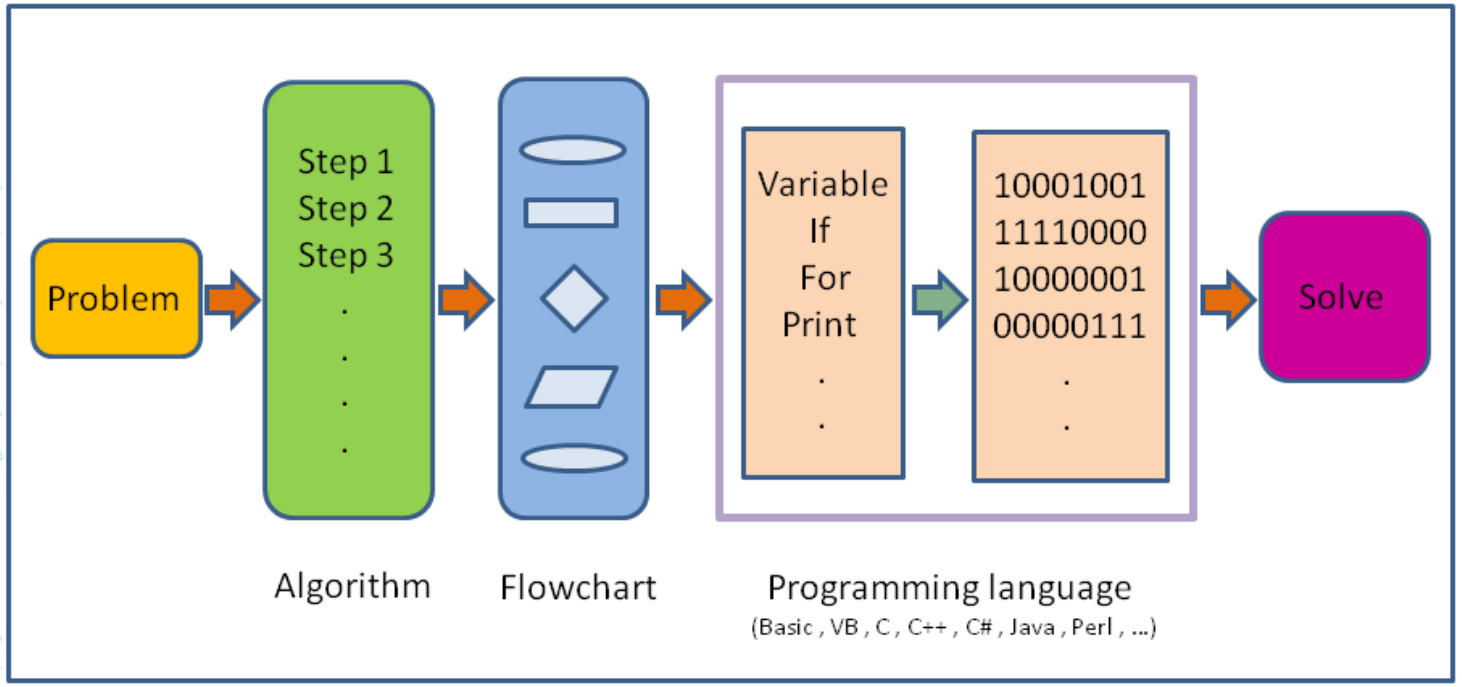
Contents

Contents

- Conditions
- Loops
- Functions
  - Functions with parameters
- Important Libraries
  - NumPy
  - Matplotlib
  - Pandas
  - Seaborn
  - SciPy

Introduction To Programming

- Programming is the process of creating a set of instructions that tell a computer how to perform a task. The instructions, known as code, are written in programming languages such as Python, Java, or C++. Let's break down the essential concepts.



What is Python?

- Python is an interpreted, object-oriented, high-level programming language with dynamic semantics.

- Its high-level built in data structures, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together.

Why Python?

- Python has emerged as the go-to language for data analysis and data science due to several factors:
  - **Versatility**: Python can handle a wide range of tasks, from simple scripting to complex data manipulation and machine learning.
  - **Rich Ecosystem**: Python boasts a vast collection of libraries specialized for data analysis, such as Pandas, NumPy, and Matplotlib, making data manipulation and visualization convenient.
  - **Community Support**: Python has a large and active community of developers, which means extensive documentation, tutorials, and community-driven resources are readily available.
- Python's flexibility and accessibility make it an ideal choice for beginners venturing into the world of data analysis.

Python installation (Windows OS)

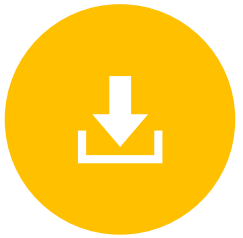
Steps:

- Go to www.python.org on your browser.
- Navigate to the “downloads” tab and click python for (Windows) OS .
- Click on “Latest Python 3 Release - Python 3.x.x”.
- Choose “Windows installer (64-bit)” at the end of the webpage
- Navigate to the downloaded files and open the installer
- It’s a must to check the red box to add the python Path to the Environment variables of your system.
- In the “Advanced Options” section, check “Install for all users” option for multi users' access.

Anaconda installation (Windows OS)

○ Go to www.Anaconda.Com on your browser.

○ Navigate to the “products” tab and click “individual edition”.



○ Click on “download” to download the windows installer.

○ Navigate to the downloaded files open the installer.

Additional Notes:

○ Make sure the you have sufficient free disk space on the drive you are installing in.

○ Check add Anaconda in my PATH environment variables in advanced options section.

Jupyter Notebook

- Jupyter Notebook is a free, open-source web application that allows users to create and share documents containing live code, equations, visualizations, and narrative text. It is commonly used for tasks such as data cleaning, transformation, numerical simulations, statistical modeling, data visualization, and machine learning, as well as serving as an educational tool.

- Google Colab, built on Jupyter notebooks, provides a robust platform hosted on Google's servers. It offers access to free GPUs and is integrated with key libraries like PyTorch, TensorFlow, Keras, and OpenCV. With Colab, there's no need for installation, and your work is automatically saved to your Google Drive account.

Virtual Working Environment

Google Colab, short for Collaboratory, is a free cloud-based platform offered by Google that allows you to write and run Python code directly in a web browser. It is designed to simplify coding, especially for students, researchers, and developers working on machine learning and data science projects.

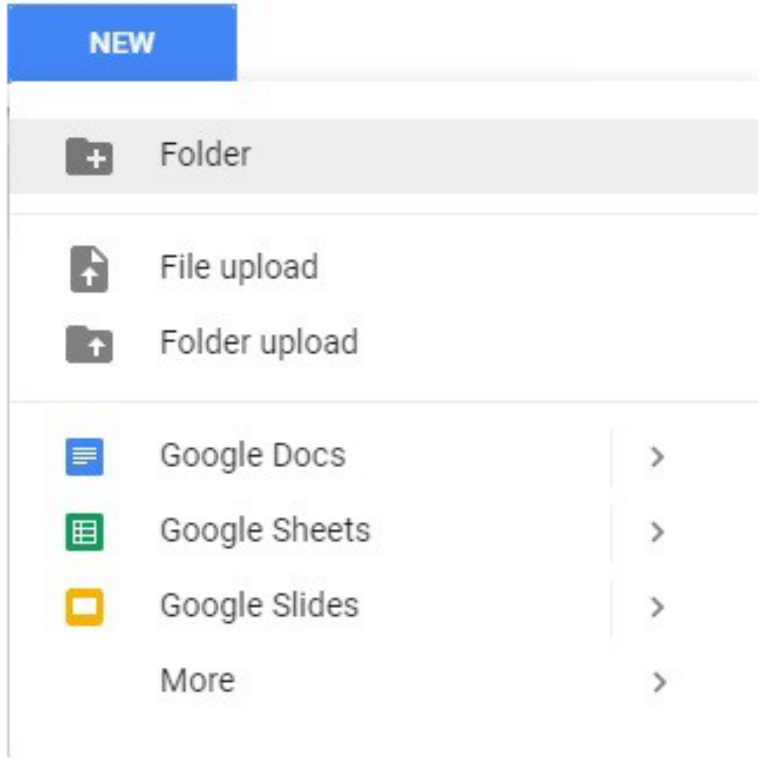
- **Cloud-Based Environment**: Colab provides a virtual workspace, allowing you to run Python code without installing anything on your local machine, as all computations are handled by Google's servers.
- **Notebooks**: Colab uses "notebooks," interactive documents that can include both code and descriptive text. This makes it easy to combine code, explanations, and visualizations in a single, interactive format.

Colab Features

- **Python Environment**: Colab supports Python, one of the most widely used programming languages for data analysis and machine learning.
- **Pre-installed Libraries**: It comes with many commonly used libraries like TensorFlow, PyTorch, and NumPy, making it highly convenient for machine learning projects.
- **Interactive Code**: You can write and run individual code cells, allowing for quick testing, debugging, and iterative development.
- **Rich Text**: You can include formatted text, images, and equations in your notebook, providing context and explanations alongside your code.
- **Visualizations**: Colab supports inline plotting, enabling you to display graphs and charts directly within your notebook.
- **Data Storage**: You can upload datasets to your Google Drive, making it easy to access and use them in your Colab notebooks.
- **Collaboration**: Being cloud-based, Colab allows you to easily share your notebooks with others, facilitating seamless collaboration.

Getting Started with Colab:

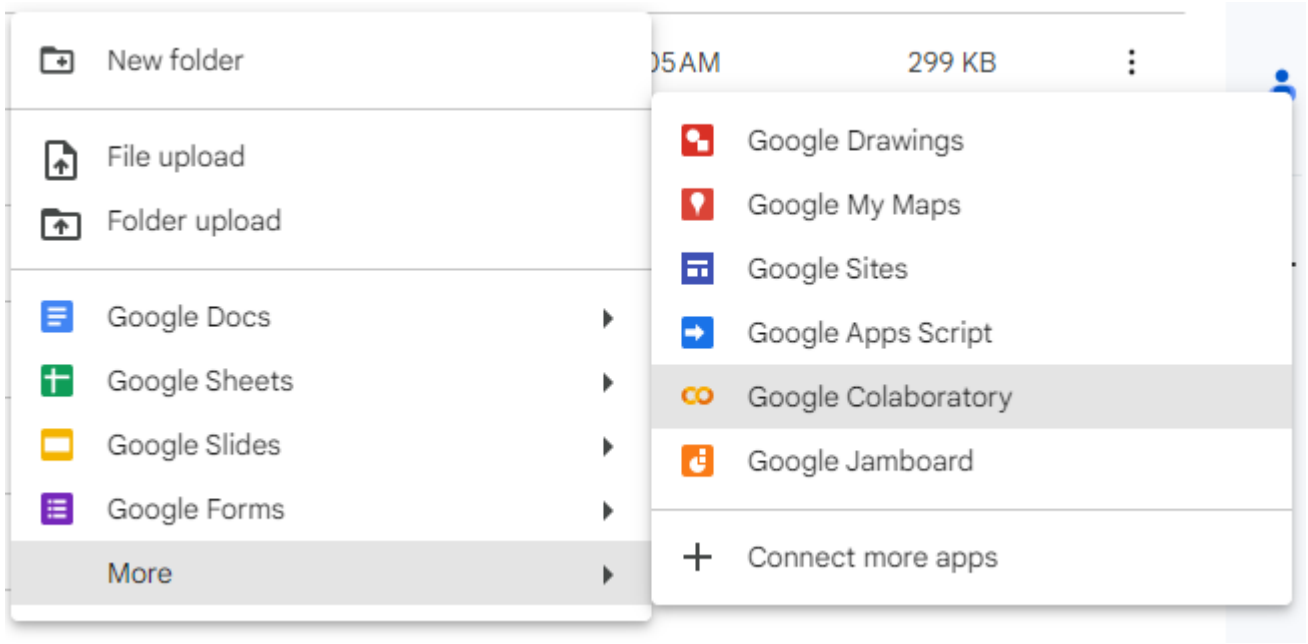
To start using Colab, you'll need a Google account (Gmail).
Creating a Folder on Google Drive:
Since Colab operates within your Google Drive, you first need to specify the folder you'll be working in. You can either create a new folder with a custom name or use the default "Colab Notebooks" folder provided by Google Drive.



Getting Started with Colab

Creating New Colab Notebook

Create a new notebook by **Right click** > **More** > **Colaboratory**



Another way :
go to the Google Colab website. And create a new notebook from scratch or upload existing notebooks.
Write your Python code in the code cells and use text cells to explain your code or provide documentation.

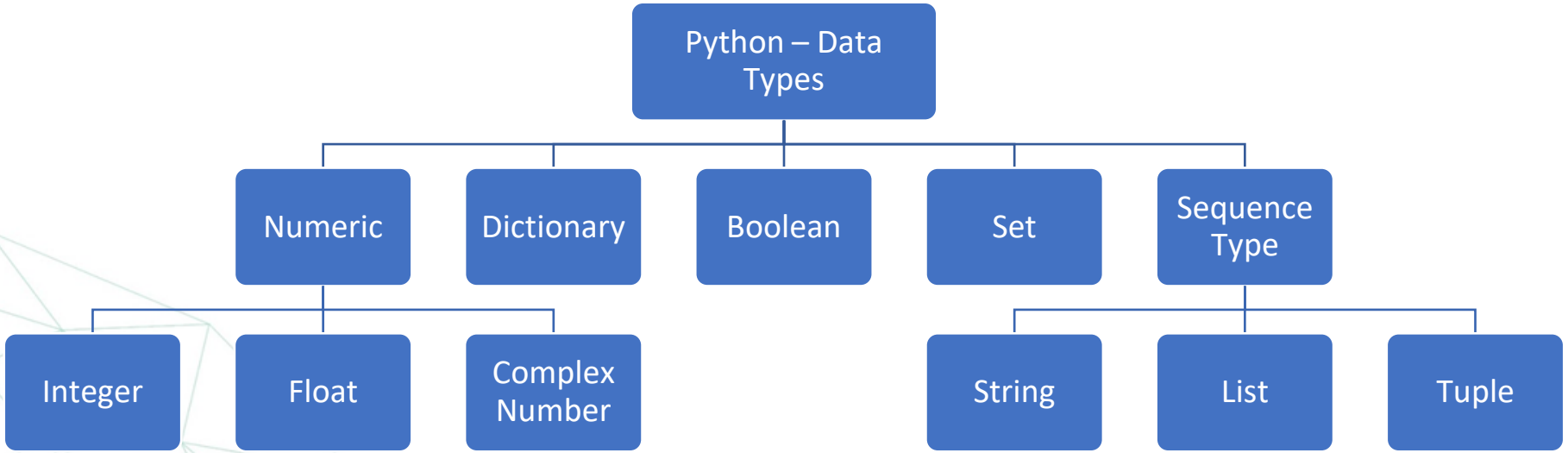
Running Code:

To run a code cell, simply click on the cell to select it. Then, press **Shift + Enter** or click the **Play** button to execute the code. The output will appear directly below the cell.

Saving and Sharing:

Colab notebooks are automatically saved to your Google Drive. You can also download your notebook in various formats (e.g., `.ipynb`, `.py`) for sharing or local storage.

Data Types


Python – Data Types
Numeric
Dictionary
Boolean
Set
Sequence Type
Integer
Float
Complex Number
String
List
Tuple

Data Types: Numeric

- **Integers** – This value is represented by int class. It contains positive or negative whole numbers (without fractions or decimals). In Python, there is no limit to how long an integer value can be.

- **Float** – This value is represented by the float class. It is a real number with a floating-point representation. It is specified by a decimal point. Optionally, the character e or E followed by a positive or negative integer may be appended to specify scientific notation.

- **Complex Numbers** – A complex number is represented by a complex class. It is specified as *(real part) + (imaginary part)j*. For example – 2+3j

Data Type	Description	Example
Integer	Whole numbers without a fractional component	`5`, `-3`, `42`
Float	Numbers with a decimal point	`5.5`, `-2.7`, `3.14`
Complex	Numbers with a real and imaginary part	`2 + 3j`, `1 - 1j`

Data Types: Sequence

String

Strings in Python are arrays of bytes representing Unicode characters. A string is a collection of one or more characters put in a single quote, double-quote, or triple-quote. In Python, there is no character data type Python, a character is a string of length one. It is represented by str class.

```
1  string = 'Hello world!'
2  print(string)
```

Data Types: Sequence

String

We can perform several operations in strings like Concatenation, Slicing, and Repetition.

Concatenation: It includes the operation of joining two or more strings together.

```
String1 = "Hello"
String2 ="World"
print(String1+String2)
Output: Hello World
```

Slicing: Slicing is a technique for extracting different parts of a string.

```
String1 = "Hello"
print(String1[2:4])
Output: llo
```

Repetition: It means repeating a sequence of instructions a certain number of times.

```
Print(String1*5)
Output: HelloHelloHelloHelloHello
```

Data Types: List

List

Lists are just like arrays, declared in other languages which is an ordered collection of data. It is very flexible as the items in a list do not need to be of the same type.

```
List = ["Ahmed", "Mohamed", "Sara"]
print("\n List containing multiple values: ")
print(List[0])
print(List[2])
```

Output:

Ahmed
Sara

Data Types: Tuple

Tuple

Just like a list, a tuple is also an ordered collection of Python objects. The only difference between a tuple and a list is that tuples are immutable i.e. tuples cannot be modified after it is created. It is represented by a tuple class.

```
# Creating a tuple
tuple1 = (1, 2, 3, 4, 5)

# Accessing elements of the tuple
print("First element of tuple")
print(tuple1[0])

print("\nLast element of tuple")
print(tuple1[-1])

print("\nThird last element of tuple")
print(tuple1[-3])
```

Output:

First element of tuple
- 1

Last element of tuple
- 5

Third last element of tuple
- 3

Data Types: Boolean

Boolean

Boolean objects that are equal to True are truthy (true), and those equal to False are falsy (false).

```
print(type(True))
print(type(False))
```

Output:

<class 'bool'>
<class 'bool'>

Data Types: Set

Set

A set is an unordered, mutable collection that can store multiple data types and is iterable. Sets automatically eliminate duplicate elements, meaning each item appears only once. While the order of elements in a set is undefined, it can contain a variety of data types.

```
set ={1, 2, 'Ahmed', 4, 'Sara',6,6}
print("\nSet with the use of Mixed Values")
print(set)
```

Output:

```
Set with the use of Mixed Values
{1, 2, 4, 6, 'Ahmed', Sara'}
```

Data Types: Dictionary

Dictionary

In Python, a dictionary is an unordered collection of data that stores key-value pairs, similar to a map. Unlike other Python data types that hold only a single value, a dictionary holds pairs of keys and values. Each key-value pair is separated by a colon :, and each pair is separated by a comma ,. This structure makes dictionaries efficient for data retrieval.

```
# Creating a dictionary
student = { "name": "John Doe", "age": 20, "courses": ["Math", "Science"], "grade": "A" }

# Accessing and printing the value associated with the key "name"
name = student["name"]
print(name)  # Output: John Doe

# Trying to get the value for the key "address"; if not found, returns "Not Available"
address = student.get("address", "Not Available")
print(address)  # Output: Not Available
```

Variables

Definition of Variables

In programming, **variables** are containers used to store data values. These values can be of various types, such as numbers, strings, lists, or other data types. Variables make it easier to label and manipulate data within a program, enhancing both its functionality and readability.

Declaring Variables in Python

In Python, **variables** are created by assigning a value to a name using the = (assignment) operator. Python is **dynamically typed**, which means you don't need to explicitly declare the data type of a variable. Instead, Python automatically infers the data type based on the value assigned.

Variable Naming Conventions

Variable names in Python can include letters, numbers, and underscores but must not start with a number. To improve code readability, it is recommended to use **descriptive and meaningful names** for variables. Variable names should be written in **lowercase letters**, with words separated by underscores (using **snake_case**).

Variables

Examples

```
age = 25
name = "John"
print(name)
print(age)
```

Output:

```
John
25
```

```
temperature_celsius = 20.5
is_raining = True
print(temperature_celsius)
print(is_raining)
```

Output:

```
20.5
True
```

Operations

- **Basic Operations in Python**
  - Python supports various types of operations, which are essential for manipulating data and controlling program flow. These operations include:

- **Arithmetic Operations:**
  - Arithmetic operations involve mathematical calculations such as addition, subtraction, multiplication, division, and exponentiation.

Example

```
x = 10
y = 5
addition_result = x + y
subtraction_result = x - y
multiplication_result = x * y
division_result = x / y
exponentiation_result = x ** y
```

Output:

```
15
5
50
2.0
100000
```

Operations

- **Comparison Operations:**
  - Comparison operations are used to compare values and determine their relationships, such as equality, inequality, greater than, less than, etc.

Example:

```
a = 10
b = 5
is_equal = a == b
is_greater_than = a > b
is_less_than_or_equal = a <= b
```

Output:

False
True
False

Operations

Logical operations are used to combine or modify Boolean values (True or False) and are fundamental in decision-making within programs. Here's an overview of the key logical operations:

- **AND**: The AND operator returns True only if both operands are True. Otherwise, it returns False.
  Example: True AND False results in False.

- **OR**: The OR operator returns True if at least one of the operands is True. It only returns False if both operands are False.
  Example: True OR False results in True.

- **NOT**: The NOT operator inverts the Boolean value of its operand. It returns True if the operand is False, and False if the operand is True.
  Example: NOT True results in False.

Example:

```
is_sunny = True
is_warm = False
is_sunny_and_warm = is_sunny and is_warm
is_sunny_or_warm = is_sunny or is_warm
```

Output:

False
True

Let’s Practice

Practice

- **Question 1:**
  - Write a Program To Calculate an are of rectangle (area = L x W)

Ans:

```
length = 10
width = 5
area = length * width
print("Area of the rectangle:", area)
```

Practice

- **Question 2:**
  - Write a program that reads the radius of a circle and calculates the area and circumference then prints the results.

Ans:

```python
import math

# Example radius
radius = 5

# Calculate area
area = math.pi * radius ** 2

# Calculate circumference
circumference = 2 * math.pi * radius

# Print the results
print("For a circle with radius", radius)
print("Area:", area)
print("Circumference:", circumference)
```

Casting

- **What is Casting?**
  - Casting refers to the process of converting a variable from one data type to another. In Python, casting allows us to change the data type of a variable as needed during program execution.
- **Types of Casting in Python**
  - There are two types of casting in Python:
    - Implicit Casting: Automatic conversion of data types by Python based on context.
    - Explicit Casting: Manual conversion of data types using built-in functions like int() , float() , str(),etc.

```
x = 10
y = 5.5
result = x + y  # Python implicitly converts x to a float before performing addition
print("Result:", result)
```

```
x = 10.5
y = int(x)  # Explicitly convert x from float to integer
print("Converted Integer:", y)
```

Taking Input From User

- In Python, you can prompt the user to enter data during program execution using the input() function.
- The input() function reads a line of text entered by the user from the keyboard and returns it as a string.

```
# Prompt the user to enter their name
name = input("Enter your name: ")

# Display a greeting message
print("Hello,", name, "!")
```

Note that the input is always of type str, which is important if you want the user to enter numbers. Therefore, you need to convert the str before trying to use it as a number

```
x = input("Write a number:")
# Out: Write a number: 10
x / 2
# Out: TypeError: unsupported operand type(s) for /: 'str' and 'int'
float(x) / 2
# Out: 5.0
```

Operations

- **Summary Question:**

Write a program that take two integers from the user and print the results of this equation:
Result = ((num1 + num2) * 3) – 10

Ans:

```
# Input two integers from the user
num1 = int(input("Enter the first integer: "))
num2 = int(input("Enter the second integer: "))

# Calculate the result of the equation
result = ((num1 + num2) * 3) - 10

# Print the result
print("Result =", result)
```

Sets

- Sets are created using curly brackets, like Set_variable = {"a", "b", 6"}.
- You can use functions like `len()` to get the number of elements and `type()` to check the data type.
- Sets can contain elements of different data types.
- You can also use the `set()` constructor instead of curly brackets to create a set.
- You cannot access elements by indexing.
- You cannot modify an element using basic assignment, such as set_variable[0] = 'c'.

Sets Notes & Methods

- **Accessing Elements**: Use the in operator to check if an element is present in the set.
- **Adding Items**: Use add() to add a single item to a set.
- **Updating Sets**: Use update() to add multiple items from another set or sequence, similar to extend() in lists.
- **Removing Items**: Use remove() to remove a specific item from the set. If the item is not present, it raises a KeyError. Use discard() to remove an item without raising an error if the item is not found.
- **Union**: union() returns a new set that is the union of two sets. It does not modify the original sets. update() performs a union but modifies the original set.
- **Intersection**: intersection() returns a new set containing only the elements present in both sets.
- **Intersection Update**: intersection_update() updates the set with the intersection of itself and another set.

Example

```
# Creating sets
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

# Accessing elements
print(2 in set1)  # Output: True

# Adding items
set1.add(7)
print("After add():", set1)
# Output: {1, 2, 3, 4, 7}

# Updating sets
set1.update([8, 9], {10})
print("After update():", set1)
# Output: {1, 2, 3, 4, 7, 8, 9, 10}
```

```
# Removing items
set1.remove(9)
print("After remove():", set1)
# Output: {1, 2, 3, 4, 7, 8, 10}

# Removing an item with discard (no error if item not found)
set1.discard(5)
print("After discard():", set1)
# Output: {1, 2, 3, 4, 7, 8, 10}

# Union of two sets
union_set = set1.union(set2)
print("Union of set1 and set2:", union_set)
# Output: {1, 2, 3, 4, 5, 6, 7, 8, 10}
```

Example

```
# Updating set1 with union (modifies set1)
set1.update(set2)
print("After update() with set2:", set1)
# Output: {1, 2, 3, 4, 5, 6, 7, 8, 10}

# Intersection of two sets
intersection_set = set1.intersection(set2)
print("Intersection of set1 and set2:", intersection_set)
# Output: {3, 4, 5, 6}

# Updating set1 with intersection (modifies set1)
set1.intersection_update(set2)
print("After intersection_update() with set2:", set1)
# Output: {3, 4, 5, 6}
```

Dictionaries

- Syntax: Dict_variable = {“name”: “Merna”, “age”: 20, 1: [1,2,3]}
- You can apply len(), type().
- It can contain different data types.
- You can use dict() constructor instead of the curly brackets.
- You can access them using Keys.
- You can modify an item using basic assignment: dict_variable[‘name’]= ‘Ahmed’

DICTIONARY Notes & Methods

- Dictionaries can be deleted using the del function in python.
- Duplicate keys are not allowed. Last key will be assigned while others are ignored. important
- built-in functions:
- .clear() to clear all elements of the dictionary.
- .copy() to copy all elements of the dictionary to another variable.
- .fromkeys() to create another dictionary with the same keys.
- .get(key) to get the values corresponding to the passed key.
- .has_key() to return True if this key is in the dictionary.
- .items() to return a list of dictionary (key, value) tuple pairs.
- .keys() to return list of dictionary dictionary keys.
- .values() to return list of dictionary dictionary values.
- .update(dict) to add key-value pairs to an existing dictionary.

Example

```
# Creating a dictionary
my_dict = {
    "name": "Alice",
    "age": 25,
    "city": "New York"
}

# Deleting the dictionary
del my_dict

# Re-creating a dictionary to demonstrate the functions
my_dict = {
    "name": "Alice",
    "age": 25,
    "city": "New York"
}
```

```
# Clearing all elements of the dictionary
my_dict.clear()
print("Dictionary after clear():", my_dict)  # Output: {}

# Copying all elements to another variable
my_dict = {
    "name": "Alice",
    "age": 25,
    "city": "New York"
}
dict_copy = my_dict.copy()
print("Copied dictionary:", dict_copy)
# Output: {'name': 'Alice', 'age': 25, 'city': 'New York'}
```

Example

```
# Creating a dictionary with the same keys but new values
keys = ["name", "age", "city"]
new_dict = dict.fromkeys(keys, "unknown")
print("Dictionary fromkeys():", new_dict)
# Output: {'name': 'unknown', 'age': 'unknown', 'city': 'unknown'}

# Getting the value corresponding to a key
value = my_dict.get("name")
print("Value for 'name':", value)  # Output: Alice

# Check if a key exists (note: has_key() is not available in Python 3)
key_exists = "age" in my_dict
print("Is 'age' key present?:", key_exists)  # Output: True
```

Example


```
# Getting a list of (key, value) tuple pairs
items = my_dict.items()
print("Items in the dictionary:", list(items))
# Output: [('name', 'Alice'), ('age', 25), ('city', 'New York')]

# Getting a list of dictionary keys
keys = my_dict.keys()
print("Keys in the dictionary:", list(keys))
# Output: ['name', 'age', 'city']

# Getting a list of dictionary values
values = my_dict.values()
print("Values in the dictionary:", list(values))
# Output: ['Alice', 25, 'New York']
```

```
# Adding key-value pairs to the existing dictionary
additional_info = {"email": "alice@example.com", "phone": "123-456-7890"}
my_dict.update(additional_info)
print("Dictionary after update():", my_dict)
# Output: {'name': 'Alice', 'age': 25, 'city': 'New York', 'email': 'alice@example.com', 'phone': '123-456-7890'}
```

Comparison between List, Tuple, Set, Dictionary

List	Tuple	Set	Dictionary
Ordered	Ordered	Unordered	Ordered
Changeable	Unchangeable	Changeable	Changeable
Duplicates yes	Duplicates yes	No Duplicates	No Duplicates
Indexed	Indexed	Unindexed	Indexed with key

Comparison between the list, Tuple, Set, Dictionary

- Ordered means that each element won’t change its place until you modify it.
- Changeable means you can edit its element.
- No Duplicates means that it only contains unique values.
- Indexed means you can access each element by its index/position except dictionaries you access elements using keys.

Conditions


```python
# Checking for Equality
string1 = "Hello World"
string2 = "Hello World"
string3 = "hello world"

# Check if strings are equal
print(string1 == string2)  # Output: True
print(string1 == string3)  # Output: False

# Ignoring Case When Checking for Equality
# Convert both strings to the same case (lowercase in this example) before comparison
print(string1.lower() == string3.lower())  # Output: True

# Alternatively, use casefold() for a more aggressive case-insensitive comparison
print(string1.casefold() == string3.casefold())  # Output: True
```

Conditions

```
# Checking for Inequality
string1 = "Hello"
string2 = "World"
string3 = "hello"

# Check if strings are not equal
print(string1 != string2)
# Output: True
print(string1 != string3)
# Output: True (case-sensitive comparison)

# Ignoring Case When Checking for Inequality
print(string1.lower() != string3.lower())
# Output: False (case-insensitive comparison)
```

```
# Numerical Comparisons
num1 = 10
num2 = 20
num3 = 10

# Check if numbers are equal
print(num1 == num2)  # Output: False
print(num1 == num3)  # Output: True

# Check if numbers are not equal
print(num1 != num2)  # Output: True
print(num1 != num3)  # Output: False

# Check if one number is greater than or less than another
print(num1 > num2)   # Output: False
print(num1 < num2)   # Output: True

# Check if one number is greater than or equal to another
print(num1 >= num3)  # Output: True

# Check if one number is less than or equal to another
print(num1 <= num2)  # Output: True
```

Conditions

```
# Define variables
age = 25
temperature = 20
is_raining = False

# Checking multiple conditions with AND
if age > 18 and temperature > 15:
    print("You are an adult and it's warm enough.")

# Checking multiple conditions with OR
if age < 18 or temperature < 10:
    print("You are either underage or it's very cold.")

# Checking multiple conditions with NOT
if not is_raining:
    print("It's not raining. You can go outside.")
```

```
# Combining multiple conditions with AND and OR
if (age > 18 and temperature > 15) or is_raining:
    print("Either you are an adult and it's warm, or it's raining.")

# Nested conditions
if age > 18:
    if temperature > 15:
        print("You are an adult and the weather is warm.")
    else:
        print("You are an adult but it's not warm.")
else:
    print("You are underage.")
```

Conditions

```
# Define a list
my_list = [1, 2, 3, 4, 5, "apple", "banana"]

# Checking whether a value is in a list
value1 = 3
value2 = "grape"

if value1 in my_list:
    print(f"{value1} is in the list.")
else:
    print(f"{value1} is not in the list.")

if value2 in my_list:
    print(f"{value2} is in the list.")
else:
    print(f"{value2} is not in the list.")
```

```
# Checking whether a value is not in a list
value3 = "pear"
value4 = 4

if value3 not in my_list:
    print(f"{value3} is not in the list.")
else:
    print(f"{value3} is in the list.")

if value4 not in my_list:
    print(f"{value4} is not in the list.")
else:
    print(f"{value4} is in the list.")
```

Conditions

If Statements

```
# Example of a simple if statement
temperature = 30

if temperature > 25:
    print("It's a hot day.")
```

If-else Statements

```
# Example of if-else statement
temperature = 18

if temperature > 25:
    print("It's a hot day.")
else:
    print("It's not a hot day.")
```

Conditions

The if-elif-else Chain

```
# Example of if-elif-else statement
temperature = 10

if temperature > 30:
    print("It's a very hot day.")
elif temperature > 20:
    print("It's a warm day.")
elif temperature > 10:
    print("It's a cool day.")
else:
    print("It's a cold day.")
```

Conditions

Using if Statements with Lists

```
# Define a list
fruits = ["apple", "banana", "cherry"]

# Check if a value is in the list
item = "banana"

if item in fruits:
    print(f"{item} is in the list.")
else:
    print(f"{item} is not in the list.")
```

Conditions

Checking that a list is Not Empty

```
# Define a list
my_list = [1, 2, 3]

# Check if the list is not empty
if my_list:
    print("The list is not empty.")
else:
    print("The list is empty.")
```

Let’s Practice

Practice

- **Question 1:**
  - Write a program that takes three integers, and prints out the smallest number.

Ans 1:

```
# Taking input from the user
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
num3 = int(input("Enter third number: "))

# Comparing the numbers to find the smallest
smallest = num1
if num2 < smallest:
    smallest = num2
if num3 < smallest:
    smallest = num3

# Printing the smallest number
print("The smallest number is:", smallest)
```

Practice

- **Question 2:**
  - Write a program that reads a student grade percentage and prints "Excellent" if his grade is greater than or equal 85, "Very Good" for 75 or greater; "Good" for 65, "Pass" for 50, "Fail" for less than 50.

Ans 2:

```
# Taking input from the user
grade_percentage = float(input("Enter the student's grade percentage: "))

# Checking the grade and printing the corresponding level of performance
if grade_percentage >= 85:
    print("Excellent")
elif grade_percentage >= 75:
    print("Very Good")
elif grade_percentage >= 65:
    print("Good")
elif grade_percentage >= 50:
    print("Pass")
else:
    print("Fail")
```

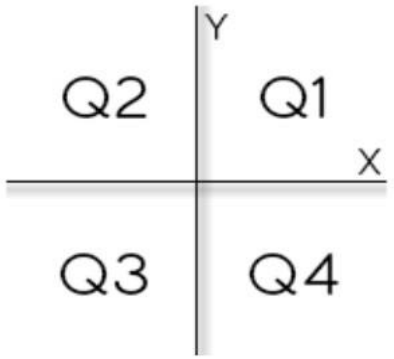
Practice

- **Question 3:**

Given two numbers X, Y which donate coordinates of a point in 2D plan. Determine in which quarter does it belong.

Note:

- Print **Q1, Q2, Q3, Q4** according to the quarter in which the point belongs to.
- Print **"Origem"** If the point is at the origin.
- Print **"Eixo X"** If the point is over X axis.
- Print **"Eixo Y"** if the point is over Y axis.



Input

Only one line containing two numbers X, Y ($-1000 \leq X, Y \leq 1000$).

Output

Print the answer to problem above.

Practice

- **Ans 3:**

```
# Taking input from the user
x = float(input("Enter the x-coordinate: "))
y = float(input("Enter the y-coordinate: "))

# Checking the position of the point
if x == 0 and y == 0:
    print("Origem")
elif x == 0:
    print("Eixo Y")
elif y == 0:
    print("Eixo X")
elif x > 0 and y > 0:
    print("Q1")
elif x < 0 and y > 0:
    print("Q2")
elif x < 0 and y < 0:
    print("Q3")
else:
    print("Q4")
```

Loops: “for” Loop

Basic “for” Loop

```
# 1. Iterating Over a List
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

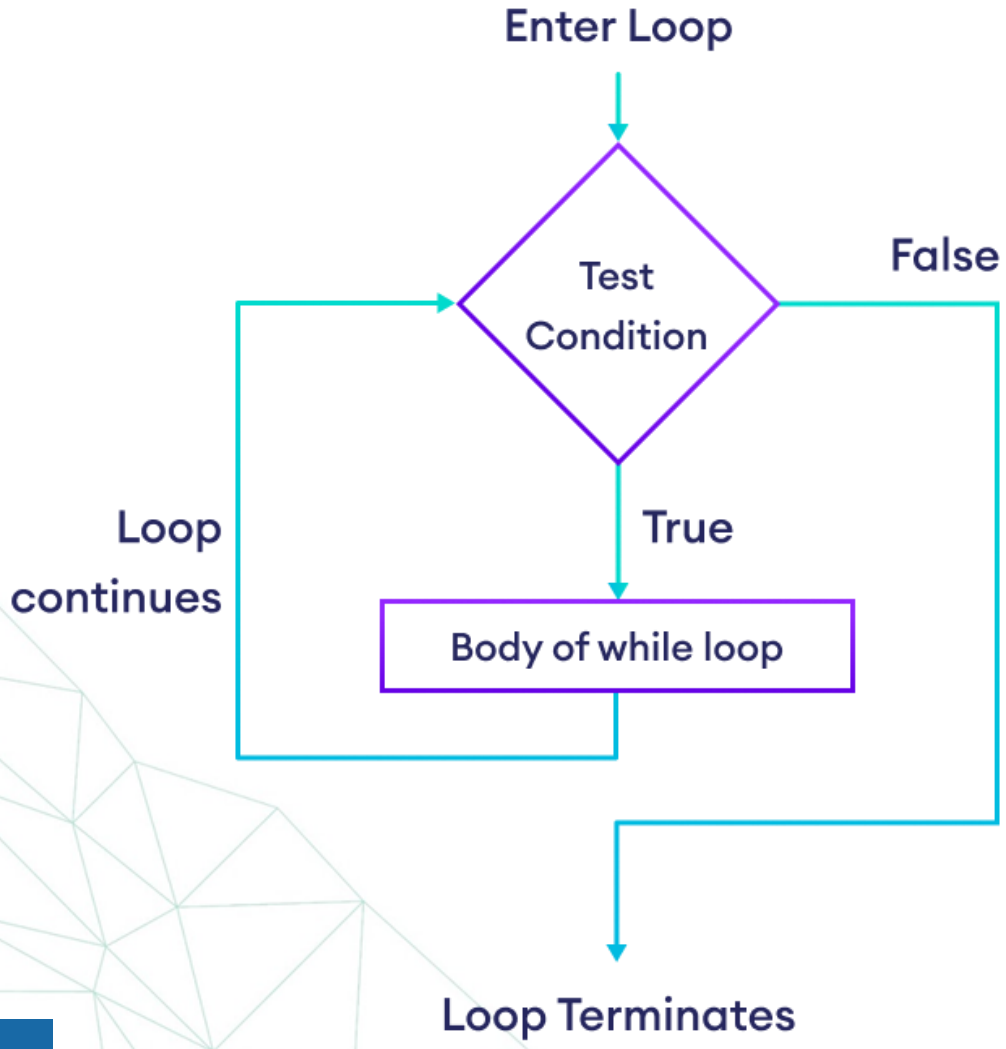
“for” Loop with range ()

```
# 3. Iterating Over a Range
for num in range(5):  # Generates numbers from 0 to 4
    print(num)
```

Iterating Through a Dictionary

```
student_scores = {
    "Alice": 85, "Bob": 92, "Charlie": 78,"David": 88
}
# Iterating Over Key-Value Pairs
for student, score in student_scores.items():
    print(f"{student}: {score}")
```

Loops


Enter Loop
Test Condition
False
Loop continues
True
Body of while loop
Loop Terminates

Loops: “while” Loop

Basic “while” Loop

```
# Initialize a counter
counter = 0

# Simple while loop
while counter < 5:
    print("Counter is:", counter)
    counter += 1  # Increment the counter
```

Loops: “while” Loop

“while” Loop with Lists & Dictionary

```
# Define the list of pets
pets = ['dog', 'cat', 'dog', 'goldfish', 'cat', 'rabbit', 'cat']

# Print the original list
print("Original list:", pets)

# While 'cat' is in the list, remove it
while 'cat' in pets:
    pets.remove('cat')
```

Loops: “while” Loop

“while” Loop with Lists & Dictionary

```
# Define the prompt message
prompt = "\nTell me something, and I will repeat it back to you:"
prompt += "\nEnter 'quit' to end the program. "

# Initialize the message variable
message = ""

# Loop until the user types 'quit'
while message != 'quit':
    message = input(prompt)  # Get user input
    print(message)  # Print the message
    if message != 'quit':
        print(message)  # Print the message again if it's not 'quit'
```

Loops: “while” Loop

“while” Loop with Dictionary

```
# Initialize an empty dictionary to store responses
responses = {}
# Set a flag to indicate that polling is active
polling_active = True
while polling_active:
    # Prompt for the person's name and response
    name = input("\nWhat is your name? ")
    response = input("Which mountain would you like to climb someday? ")
    # Store the response in the dictionary
    responses[name] = response
    # Find out if anyone else is going to take the poll
    repeat = input("Would you like to let another person respond? (yes/no) ")

    if repeat.lower() == 'no':
        polling_active = False

# Polling is complete. Show the results
print("\n--- Poll Results ---")
for name, response in responses.items():
    print(f"{name} would like to climb {response}.")
```

Let’s Practice

Practice

- **Question 1:**
  - Print sum of first 100 integers.

Ans 1:

```
# Initialize the sum variable to store the sum of integers
sum_of_integers = 0

# Loop through the integers from 1 to 100 and add them to the sum
for i in range(1, 101):
    sum_of_integers += i

# Print the sum of the first 100 integers
print("Sum of the first 100 integers:", sum_of_integers)
```

Practice

- **Question 2:**
  - Write a program that reads a positive integer and computes the factorial.

Ans 2:

```
# Taking input from the user
num = int(input("Enter a positive integer: "))

# Checking if the input is valid (positive integer)
if num < 0:
    print("Factorial is not defined for negative numbers.")
else:
    # Initializing factorial to 1
    factorial = 1

    # Computing factorial using a loop
    for i in range(1, num + 1):
        factorial *= i

    # Printing the factorial
    print("Factorial of", num, "is:", factorial)
```

Practice

- **Question 3:**
  - Write a program that given a number *N*. Print all **even** numbers between **1** and ***N*** inclusive in separate lines.

Ans 3:

```
# Taking input from the user
N = int(input("Enter a number: "))

# Printing even numbers between 1 and N inclusive using if condition
print("Even numbers between 1 and", N, "inclusive:")
for i in range(1, N + 1):
    if i % 2 == 0:
        print(i)
```

Functions

Defining and Calling Function

```
def greet():
    print("Hello, world!")
# Calling the function
greet()
```

Function with Parameters

```
# Function with parameters
def greet_person(name):
    print(f"Hello, {name}!")

# Calling the function with an argument
greet_person("Alice")
```

Functions

Function with Multiple Parameters

```
# Function with multiple parameters
def add(a, b):
    return a + b
# Calling the function with arguments
result = add(3, 5)
print(result)
```

Default Parameter Values

```
# Function with default parameter values
def greet(name="Guest"):
    print(f"Hello, {name}!")
# Calling the function without an argument
greet()
# Calling the function with an argument
greet("Bob")
```

Functions

Keyword Argument

```
# Function with keyword arguments
def describe_person(name, age, city):
    print(f"{name} is {age} years old and lives in {city}.")

# Calling the function with keyword arguments
describe_person(name="Alice", age=30, city="New York")
```

Let’s Practice

Practice

- **Question 1:**
  - Write a function that reads the radius of a circle and calculates the area and circumference then prints the results.

Ans 1:

```python
import math

def calculate_circle_properties(radius):
    # Calculate area and circumference
    area = math.pi * radius ** 2
    circumference = 2 * math.pi * radius

    # Print the results
    print("Radius of the circle:", radius)
    print("Area of the circle:", area)
    print("Circumference of the circle:", circumference)

# Taking input from the user
radius = float(input("Enter the radius of the circle: "))

# Calling the function to calculate and print the properties
calculate_circle_properties(radius)
```

Practice

- **Question 2:** Memo and Momo are playing a game. Memo will choose a positive number a, and Momo will choose a positive number b.

  Your task is to tell them who will win according to the following rules:

  - If both a and b are divisible by k, both of them win and you should print "Both".
  - If a is divisible by k but b isn't, Memo wins and you should print "Memo".
  - If b is divisible by k but a isn't, Momo wins and you should print "Momo".
  - If both a and b are not divisible by k, no one wins and you should print "No One".

Ans 2:

```python
def determine_winner(k, a, b):
    if a % k == 0 and b % k == 0:
        return "Both"
    elif a % k == 0:
        return "Memo"
    elif b % k == 0:
        return "Momo"
    else:
        return "No One"

# Taking input for k, a, and b
k = int(input("Enter the value of k: "))
a = int(input("Enter the value of a: "))
b = int(input("Enter the value of b: "))

# Calling the function and printing the result
print(determine_winner(k, a, b))
```

Summary Question

- **Summary Question:**
  - Write a function that takes one integer and print if it is prime or not.

Ans:

```
def is_prime(num):
    if num <= 1:
        print(num, "is not a prime number.")
    elif num == 2:
        print(num, "is a prime number.")
    else:
        for i in range(2, int(num ** 0.5) + 1):
            if num % i == 0:
                print(num, "is not a prime number.")
                break
        else:
            print(num, "is a prime number.")

# Taking input from the user
number = int(input("Enter an integer: "))

# Calling the function to check if the number is prime
is_prime(number)
```

NumPy

- It’s the main library in python that deals with scientific computing, it provides high performance multi-dimensional array
- **So mainly it deals with arrays and metrices**

1. How to create an array

- First: import the library
- **np.array** which takes the numbers
- **Ex:** np.array([1,2,3,4])

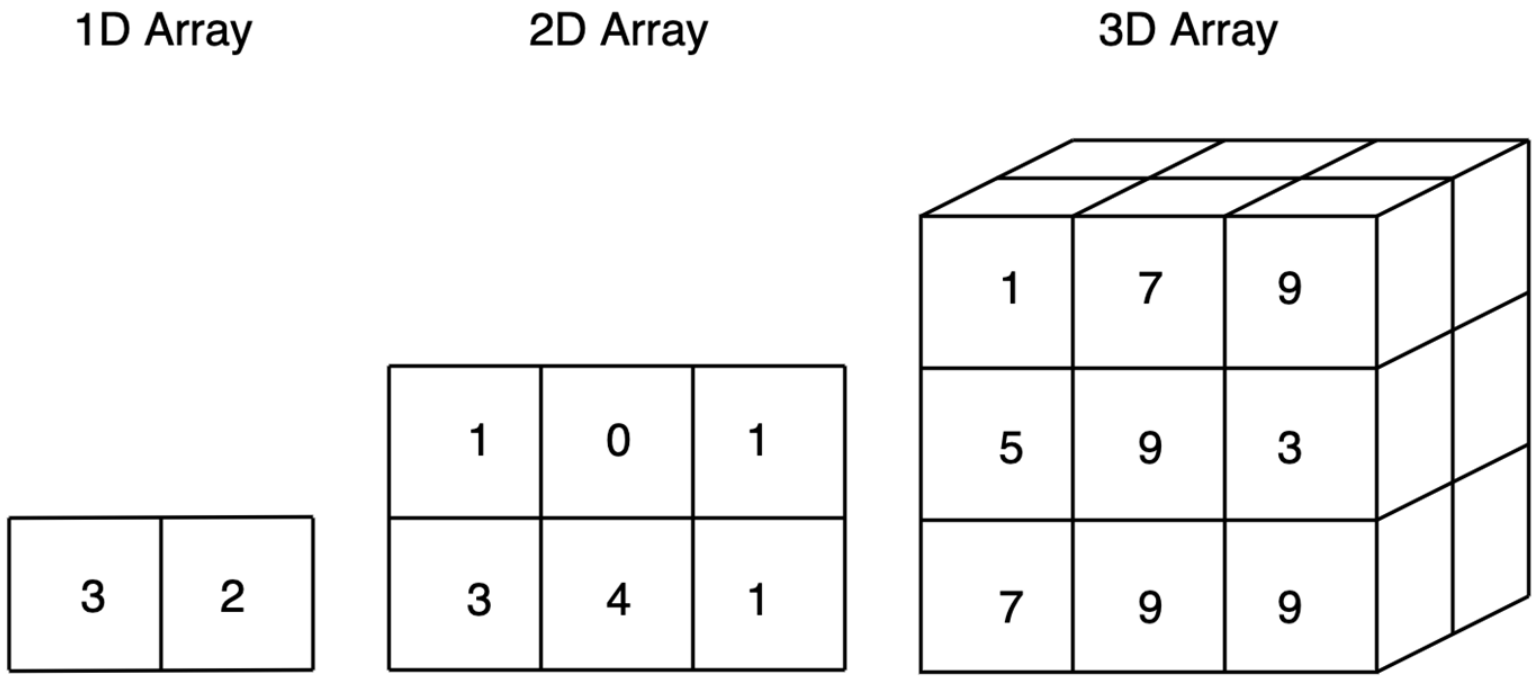
Getting the shape and type

- Arrays can be one dimensional or multidimensional so in order to know that to know that you have to get the shape
- **Array. Shape**
- **Type(array)**
- This is for getting the type of an array

Change the values in the array

- Ex: array = np.array([1,2,3,4])
- The index of the values starts from the zeroth element
- In order to access number 3 for example, then it is located at index 2
- To change it: array[2] = 7
- The new array now is [1,2,7,4]

Multi dimensional array



- Multi dimensional array means that it has more than one row and Colum
- To create it: **np.array([[10,20,30,40], [50,60,70,80]])**
- Now it is a 2d array
- To get the shape: **array. Shape**
- **(2, 4) which means 2 rows and 4 columns**

Indexing and slicing

- Ex: array = np.array([[1,2,3], [7,8,9], [3,5,7]])
- For slicing: array[rows , columns] so
- **array[0:2 , 1:3] ??**
- Note: the first one is included, the second is excluded

Creating automatic arrays

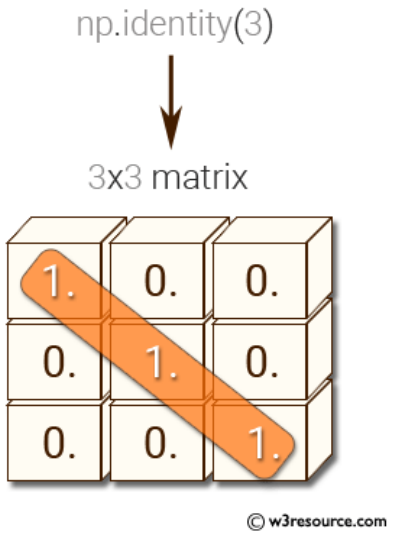
- 1. zeros array
- 2. ones array
- 3. identity array
- 4. constant array
- 5. random array

Zeros & ones & constant

- Zeros: **np.zeros((shape of the array))**
- Ones: **NP.ONES((SHAPE OF THE ARRAY))**
- Constant number: **NP.FULL((SHAPE OF THE ARRAY), (THE NUMBER))**

Identity Metrix

- To create it: **np.eye(the number of ones)**



Random Metrix

- The most important one for machine and deep learning.
- **np.random.random((shape of the array))**
- The numbers are randomly chosen and is between 0 and 1
- Used for initializing the weights in ml & dl

Argmax

- Argmax is a method that brings you the index of the highest values
- Ex: arr = np.array([0.2, 0.4, 0.8])

  print(np.argmax (arr)) index 2 is the output

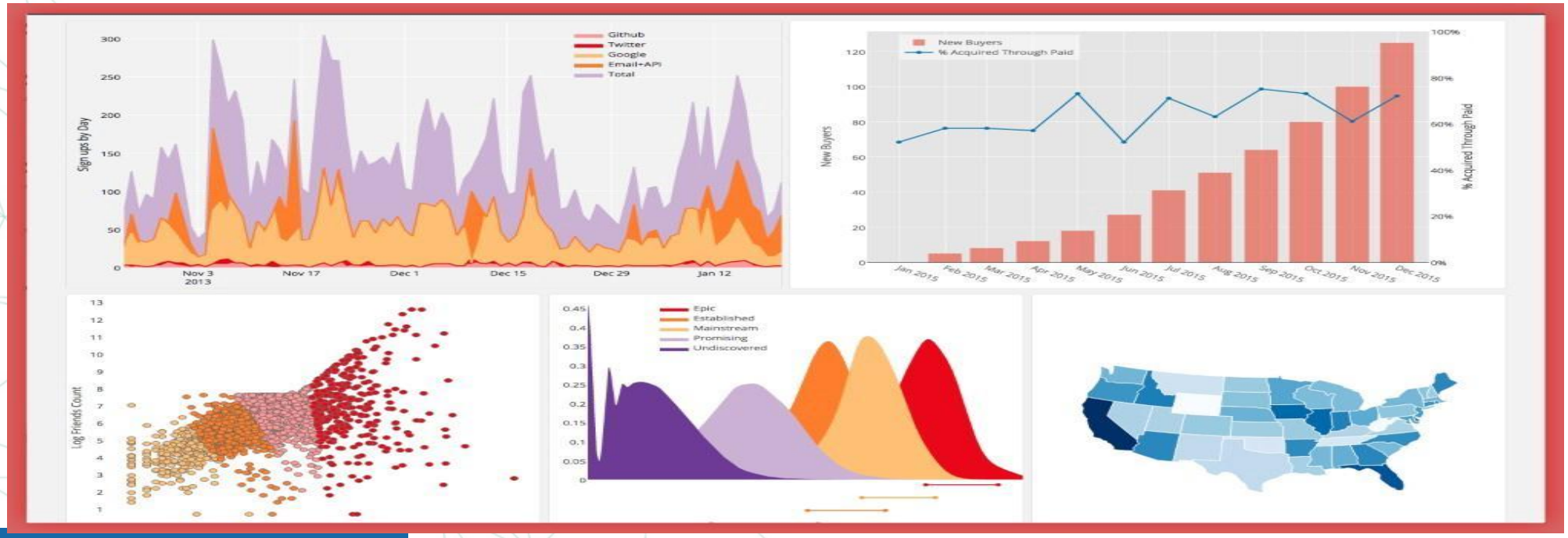
Note: this function is used mostly in deep learning (activation function outputs)

Arithmetic operations

- Np.add (x, y) where x and y are arrays
- Np.subtract (x, y)
- Np.multiply (x, y)
- Np.divide (x, y)
- Np.sqrt (x)
- Np.sin (x)
- Np.cos (x)

2. matplotlib

- Used for data visualization and plotting graphs



Steps for data visualization

- 1. create the data (array of x & array of y)
- 2. plot the graph
- 3. give title
- 4. label the axis
- 5. name each plot
- 6. show the graph

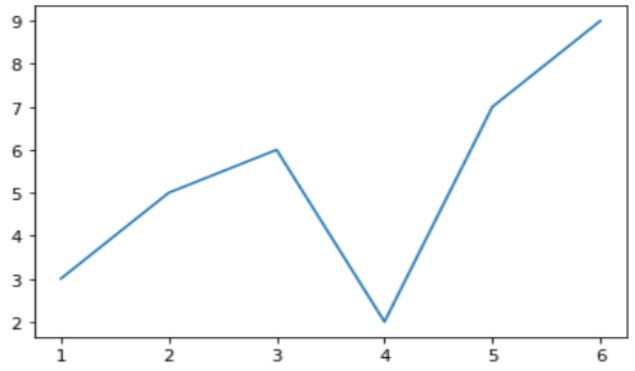
Methods of matplotlib

- 1. plt.plot
- 2. plt.scatter
- 3. plt.title
- 4. plt.legend
- 5. plt.xlable
- 6. plt.ylable
- 7. plt.subplot
- 8. plt.show

PLOT & SCATTER & TITLE

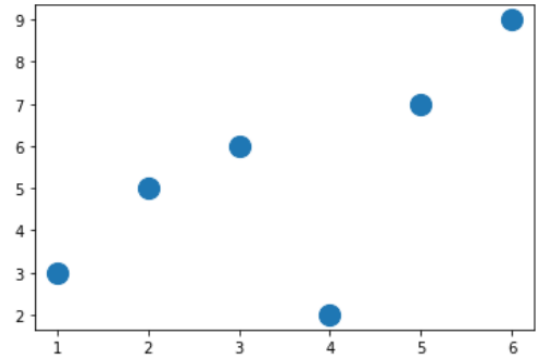
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([1,2,3,4,5,6])
y = np.array([3,5,6,2,7,9])
plt.plot(x, y)
```

```
[<matplotlib.lines.Line2D at 0x7f7f63e204a8>]
```



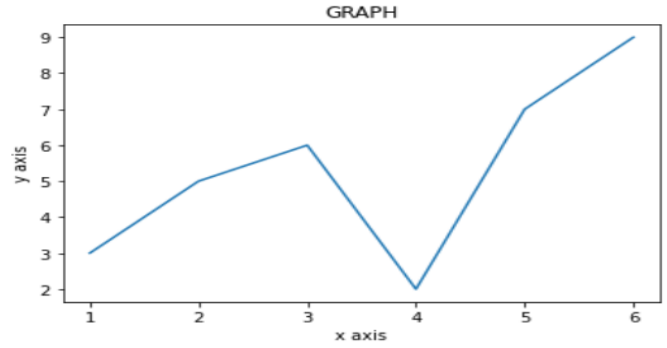
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([1,2,3,4,5,6])
y = np.array([3,5,6,2,7,9])
plt.scatter(x, y, s=200)
```

```
<matplotlib.collections.PathCollection at 0x7f7f6408c3c8>
```



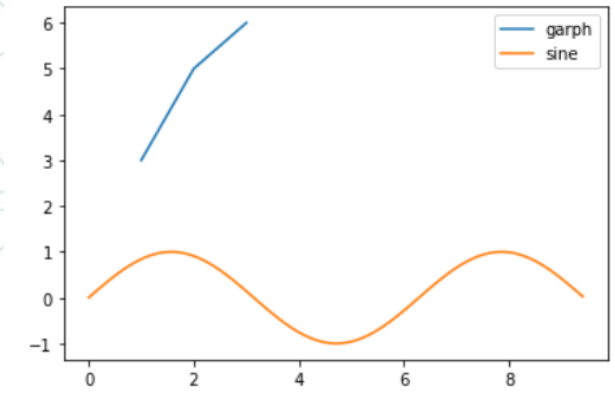
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([1,2,3,4,5,6])
y = np.array([3,5,6,2,7,9])
plt.title("GRAPH")
plt.xlabel("x axis")
plt.ylabel("y axis")
plt.plot(x, y)
```

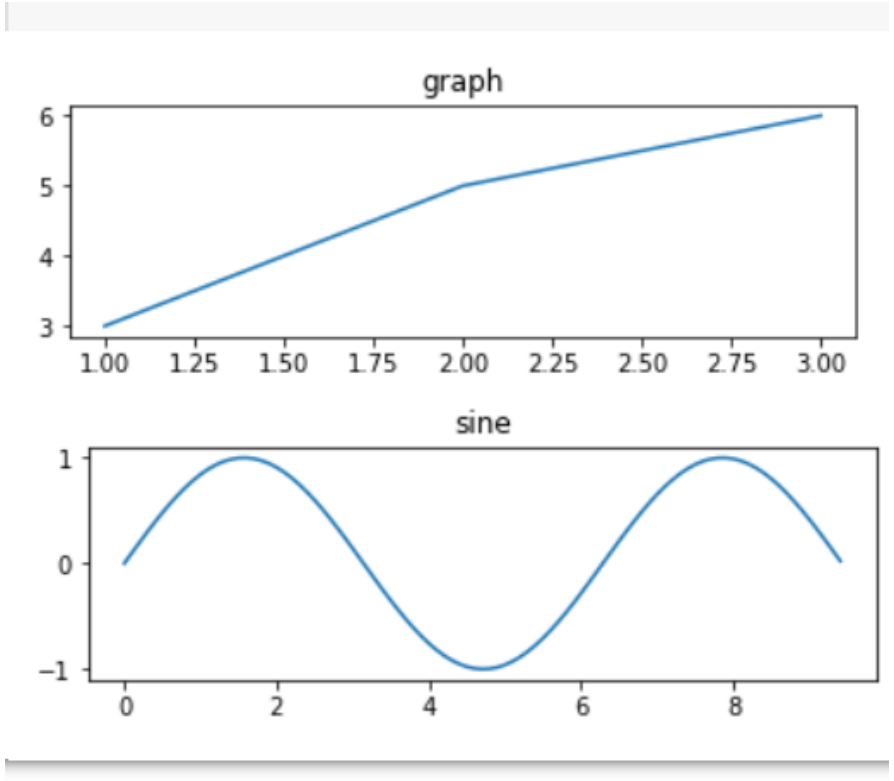
```
[<matplotlib.lines.Line2D at 0x7f7f647152e8>]
```



Legends & subplot

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([1,2,3])
y = np.array([3,5,6])
x2 = np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x2)
plt.plot(x, y)
plt.plot(x2, y_sin)
plt.legend(['garph','sine'])
plt.show()
```





subplots

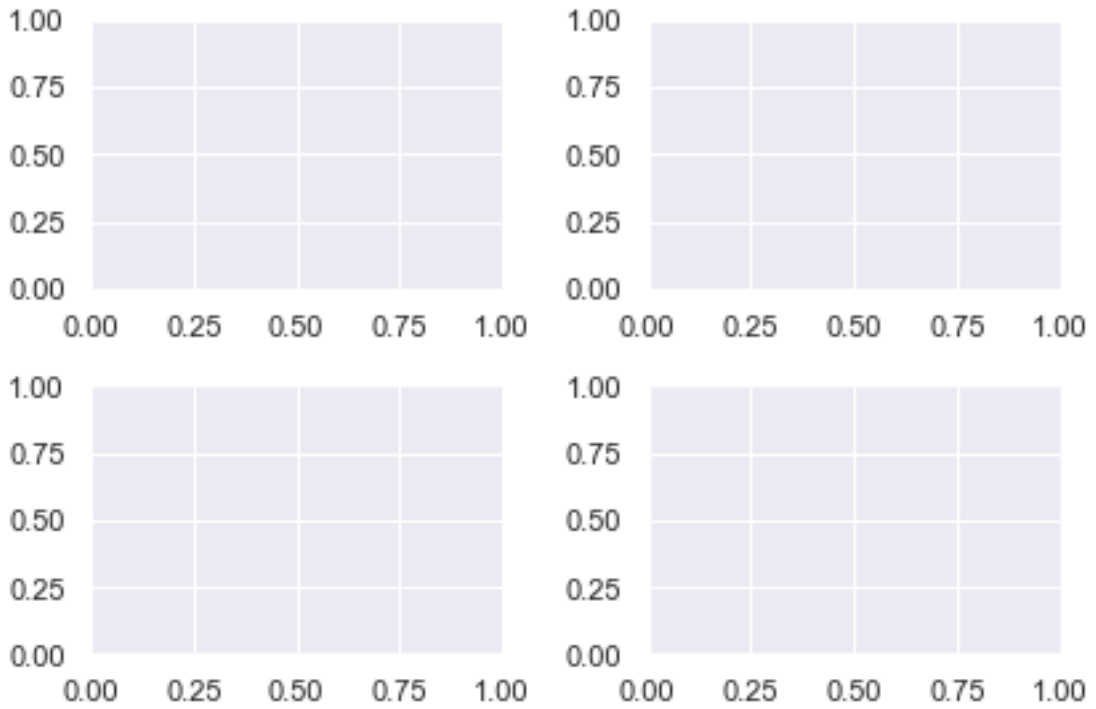
```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(1, 10, 20)
y = np.linspace(2, 30, 20)
z = np.array([1,3,4,2,2,2,2,2,5,6,7,8,9,0,9,9,8,7,8,7])

fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(15,15))

ax1.plot(x, y, color = 'r')
ax2.plot(x, z)
ax4.plot(x, y)
ax3.plot(x, z)
plt.show()
```

subplots



3. pandas

- Excel sheet for python.
- The most essential library for ml because it is where the data gets preprocessed.
- Clean the data by doing things like removing missing values and filtering rows or columns by some criteria.

Data Structures In Pandas

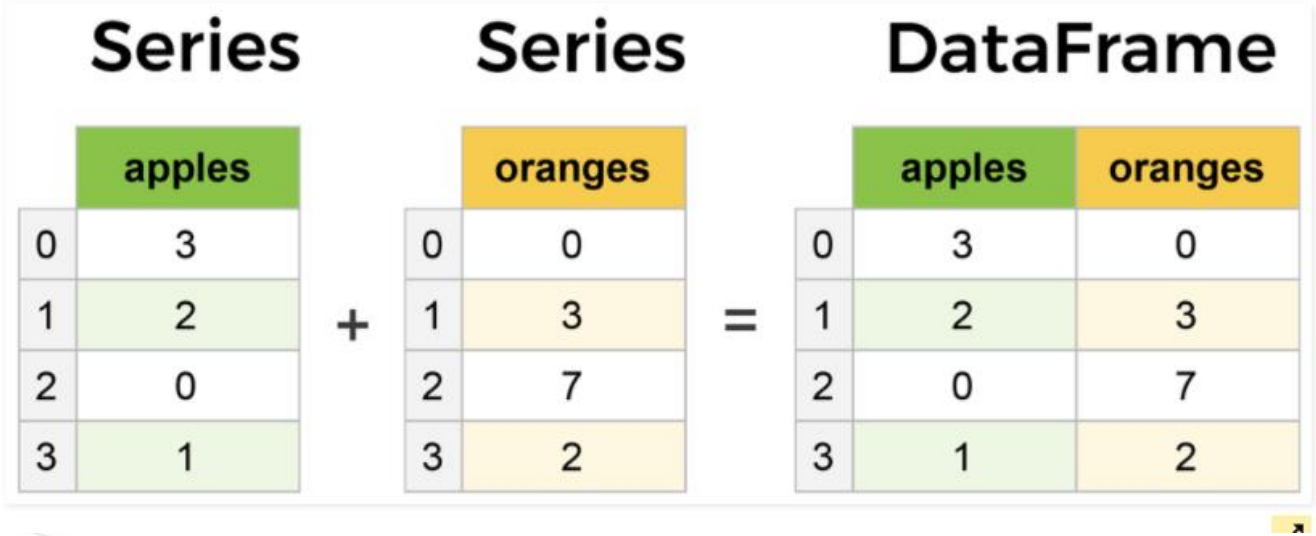
- The Pandas library deals with the following data structures :

- Pandas Data Frame :
- Whenever there is a dataset with at least two columns and any number of records(rows) then it is known as a Data frame.

- Pandas Series :
- Whenever there is a dataset with just a single column with any number of records (rows) then it is known as a Series.

Primary Components of pandas

- 1. Series
- 2. Data frame



	Series			Series			DataFrame	
	apples			oranges			apples	oranges
0	3	+	0	0	=	0	3	0
1	2		1	3		1	2	3
2	0		2	7		2	0	7
3	1		3	2		3	1	2

Importing The Pandas Library

- Before learning and using the functionalities of Pandas, it is necessary to import the Pandas library first. We do so by writing the following code into our Jupyter notebook :

- **import pandas as pd**

- Note : “pd” is used as an alias so that the Pandas package can be referred to as “pd” instead of “pandas”.

Creating data frame from scratch

```
data = {
    'apples': [3, 2, 0, 1],
    'oranges': [0, 3, 7, 2]
}
```

And then pass it to the pandas DataFrame constructor:

```
purchases = pd.DataFrame(data)

purchases
```

output

OUT:

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

Importing Data

- Importing Data
- Before working on data, we have to first import it. The Pandas library has a variety of commands for dealing with different forms of data. We will be learning about one such command which deals with CSV files**.**

- **1. read_csv()**
  - The **pd.read_csv()** command is used to read a CSV file into data frame.

Viewing/Inspecting Data with Pandas

- **1. head() and tail()**
-

  The **df.head()** command helps us view our dataset's top 5 (default value) records. If you want to view more than 5, you can do so by typing – **df.head(n)** where n is the number of records you want to view.

Viewing/Inspecting Data with Pandas

```
In [4]: df.head()
```

Out[4]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	watching
0	Fullmetal Alchemist: Brotherhood	TV	64.0	NaN	False	2009.0	2010.0	Spring	The foundation of alchemy is based on the law ...	['Bones']	['Action', 'Adventure', 'Drama', 'Fantasy', 'M...	['Animal Abuse', 'Mature Themes', 'Violence', ...	103707.0	14351
1	your name.	Movie	1.0	107.0	False	2016.0	2016.0	NaN	Mitsuha and Taki are two total strangers livin...	['CoMix Wave Films']	['Drama', 'Romance', 'Body Swapping', 'Gender ...	[]	58831.0	1453
2	A Silent Voice	Movie	1.0	130.0	False	2016.0	2016.0	NaN	After transferring into a new school, a deaf g...	['Kyoto Animation']	['Drama', 'Shounen', 'Disability', 'Melancholy...	['Bullying', 'Mature Themes', 'Suicide']	45892.0	946
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	TV	10.0	NaN	False	2016.0	2016.0	Fall	Picking up where the second season ended, the ...	['Production I.G']	['Shounen', 'Sports', 'Animeism', 'School Club...	[]	25134.0	2183
4	Attack on Titan 3rd Season: Part II	TV	10.0	NaN	False	2019.0	2019.0	Spring	The battle to retake Wall Maria begins now! Wi...	['Wit Studio']	['Action', 'Fantasy', 'Horror', 'Shounen', 'Da...	['Cannibalism', 'Explicit Violence']	21308.0	3217

- The **df.tail()** command helps us view our dataset's last 5 (default value) records. If you want to view more than 5, you can do so by typing – **df.tail(n)** where n is the number of records you want to view

shape

- The **df.shape** command provides you with the number of rows and columns in your dataset.
-

```
In [7]: df.shape

Out[7]: (14578, 18)
```

Info()

- The **df.info()** command prints the information about our dataset, including columns, datatypes of columns, non-null values, and memory usage.

```
In [8]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14578 entries, 0 to 14577
Data columns (total 18 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   title          14578 non-null  object
 1   mediaType      14510 non-null  object
 2   eps            14219 non-null  float64
 3   duration       9137 non-null   float64
 4   ongoing        14578 non-null  bool
 5   startYr        14356 non-null  float64
 6   finishYr       14134 non-null  float64
 7   sznOfRelease   3767 non-null   object
 8   description    8173 non-null   object
 9   studios        14578 non-null  object
 10  tags           14578 non-null  object
 11  contentWarn    14578 non-null  object
 12  watched        14356 non-null  float64
 13  watching       14578 non-null  int64
 14  wantWatch      14578 non-null  int64
 15  dropped        14578 non-null  int64
 16  rating         12107 non-null  float64
 17  votes          12119 non-null  float64
dtypes: bool(1), float64(7), int64(3), object(7)
memory usage: 1.9+ MB
```

Describe

- The **df.describe()** command calculates a summary of statistics for the data frame columns. This function returns the count, mean, standard deviation, and interquartile range (IQR) values.

```
In [9]: df.describe()
```

Out[9]:

	eps	duration	startYr	finishYr	watched	watching	wantWatch	dropped	rating	votes
count	14219.000000	9137.000000	14356.000000	14134.000000	14356.000000	14578.000000	14578.000000	14578.000000	12107.000000	12119.000000
mean	13.501231	23.465142	2005.457788	2005.515919	2408.043396	213.026684	1021.729112	125.963026	2.948697	2085.787771
std	62.262185	30.777048	14.707105	14.656509	7168.368428	1261.707640	2145.010604	453.577348	0.827642	5946.283685
min	1.000000	1.000000	1907.000000	1907.000000	0.000000	0.000000	0.000000	0.000000	0.844000	10.000000
25%	1.000000	3.000000	2000.000000	2000.000000	25.000000	1.000000	24.000000	1.000000	2.303500	34.000000
50%	1.000000	8.000000	2010.000000	2010.000000	165.000000	7.000000	175.000000	7.000000	2.965000	218.000000
75%	12.000000	29.000000	2016.000000	2016.000000	1469.500000	63.000000	980.000000	40.000000	3.615500	1412.500000
max	2527.000000	235.000000	2026.000000	2026.000000	161567.000000	74537.000000	28541.000000	19481.000000	4.702000	131067.000000

nunique

- The **df.nunique** command returns the number of unique entries in each column.

```
In [13]: df.nunique()

Out[13]: title          14578
         mediaType          8
         eps              209
         duration         147
         ongoing            2
         startYr          104
         finishYr         104
         sznOfRelease       4
         description     8108
         studios          864
         tags            9580
         contentWarn      178
         watched         4317
         watching        1463
         wantWatch       3521
         dropped         1232
         rating          3306
         votes           3787
         dtype: int64
```

Selection of Data

- You often do not want to work with only a subset of the entire dataset. In such cases, use the following commands:
- The **df[col]** command returns the column with the specified label as series.

```
In [61]: df['title']

Out[61]: 0                        Fullmetal Alchemist: Brotherhood
         1                                                your name.
         2                                             A Silent Voice
         3        Haikyuu!! Karasuno High School vs Shiratorizaw...
         4                         Attack on Titan 3rd Season: Part II
                                          ...
         14573                Welcome to Demon School, Iruma-kun 2
         14574       Pinocchio-P: Sekai wa Mada Hajimatte Sura Inai
         14575                                             Minagoroshi
         14576                                             Kurayukaba
         14577                                          YOASOBI: Harujion
         Name: title, Length: 14578, dtype: object
```

Selection of Data

- **df[[col1,col2]]**
- The **df[[col1, col2]]** returns columns as a data frame.
-

```
In [62]: df[['title', 'mediaType']]
```

Out[62]:

	title	mediaType
0	Fullmetal Alchemist: Brotherhood	TV
1	your name.	Movie
2	A Silent Voice	Movie
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	TV
4	Attack on Titan 3rd Season: Part II	TV
...	...	...
14573	Welcome to Demon School, Iruma-kun 2	TV
14574	Pinocchio-P: Sekai wa Mada Hajimatte Sura Inai	Music Video
14575	Minagoroshi	OVA
14576	Kurayukaba	Movie
14577	YOASOBI: Harujion	Music Video

14578 rows × 2 columns

Selection of Data

- The **df.loc[ ]** command helps you access a group of rows and columns. loc is label-based, which means that you have to specify rows and columns based on their row and column labels. It includes the ends, i.e., **df.loc[0:4]** will return all the rows from 0-4(included).

```
In [33]: df.loc[:4]
```

Out[33]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	watching
0	Fullmetal Alchemist: Brotherhood	TV	64.0	NaN	False	2009.0	2010.0	Spring	The foundation of alchemy is based on the law ...	['Bones']	['Action', 'Adventure', 'Drama', 'Fantasy', 'M...	['Animal Abuse', 'Mature Themes', 'Violence', ...	103707.0	14351
1	your name.	Movie	1.0	107.0	False	2016.0	2016.0	NaN	Mitsuha and Taki are two total strangers livin...	['CoMix Wave Films']	['Drama', 'Romance', 'Body Swapping', 'Gender ...	[]	58831.0	1453
2	A Silent Voice	Movie	1.0	130.0	False	2016.0	2016.0	NaN	After transferring into a new school, a deaf g...	['Kyoto Animation']	['Drama', 'Shounen', 'Disability', 'Melancholy...	['Bullying', 'Mature Themes', 'Suicide']	45892.0	946
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	TV	10.0	NaN	False	2016.0	2016.0	Fall	Picking up where the second season ended, the ...	['Production I.G']	['Shounen', 'Sports', 'Animeism', 'School Club...	[]	25134.0	2183
4	Attack on Titan 3rd Season: Part II	TV	10.0	NaN	False	2019.0	2019.0	Spring	The battle to retake Wall Maria begins now! Wi...	['Wit Studio']	['Action', 'Fantasy', 'Horror', 'Shounen', 'Da...	['Cannibalism', 'Explicit Violence']	21308.0	3217

Selection of Data

selecting all the rows and the column named 'title’

```
In [22]: df.loc[:,'title']

Out[22]: 0                        Fullmetal Alchemist: Brotherhood
         1                                                your name.
         2                                             A Silent Voice
         3        Haikyuu!! Karasuno High School vs Shiratorizaw...
         4                      Attack on Titan 3rd Season: Part II
                                          ...
         14573                Welcome to Demon School, Iruma-kun 2
         14574       Pinocchio-P: Sekai wa Mada Hajimatte Sura Inai
         14575                                            Minagoroshi
         14576                                             Kurayukaba
         14577                                         YOASOBI: Harujion
         Name: title, Length: 14578, dtype: object
```

Selection of Data

selecting all the rows from 1-5 and the columns named 'title' and 'rating'

```
In [26]: df.loc[1:5,['title','rating']]
```

Out[26]:

	title	rating
1	your name.	4.663
2	A Silent Voice	4.661
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	4.660
4	Attack on Titan 3rd Season: Part II	4.650
5	Demon Slayer: Kimetsu no Yaiba	4.647

Selection of Data

selecting all the rows and columns with entries having rating > 4.5

```
In [38]: df.loc[df['rating'] > 4.5]
```

Out[38]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	wa
0	Fullmetal Alchemist: Brotherhood	TV	64.0	NaN	False	2009.0	2010.0	Spring	The foundation of alchemy is based on the law ...	['Bones']	['Action', 'Adventure', 'Drama', 'Fantasy', 'M...	['Animal Abuse', 'Mature Themes', 'Violence', ...	103707.0	
1	your name.	Movie	1.0	107.0	False	2016.0	2016.0	NaN	Mitsuha and Taki are two total strangers livin...	['CoMix Wave Films']	['Drama', 'Romance', 'Body Swapping', 'Gender ...	[]	58831.0	
2	A Silent Voice	Movie	1.0	130.0	False	2016.0	2016.0	NaN	After transferring into a new school, a deaf g...	['Kyoto Animation']	['Drama', 'Shounen', 'Disability', 'Melancholy...	['Bullying', 'Mature Themes', 'Suicide']	45892.0	
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	TV	10.0	NaN	False	2016.0	2016.0	Fall	Picking up where the second season ended, the ...	['Production I.G']	['Shounen', 'Sports', 'Animeism', 'School Club...	[]	25134.0	
4	Attack on Titan 3rd Season: Part II	TV	10.0	NaN	False	2019.0	2019.0	Spring	The battle to retake Wall Maria begins now! Wi...	['Wit Studio']	['Action', 'Fantasy', 'Horror', 'Shounen', 'Da...	['Cannibalism', 'Explicit Violence']	21308.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
93	Youjo Senki: Saga of Tanya the Evil Movie	Movie	1.0	98.0	False	2019.0	2019.0	NaN	The time is UC 1926. The Imperial Army's	['Studio NUT']	['Action', 'Fantasy', 'Guns', 'Isekai', 'Milit...	['Violence']	4272.0	

Selection of Data

- The **df.iloc[ ]** command also helps you access a group of rows and columns. Still, unlike loc, iloc is integer position-based, so you have to specify rows and columns by their integer position values. It excludes the ends, i.e., **df.iloc[0:4]** will return all the rows from 0-3 as 4 is excluded.

```
In [55]: df.iloc[0:4]
```

Out[55]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	watching
0	Fullmetal Alchemist: Brotherhood	TV	64.0	NaN	False	2009.0	2010.0	Spring	The foundation of alchemy is based on the law ...	['Bones']	['Action', 'Adventure', 'Drama', 'Fantasy', 'M...	['Animal Abuse', 'Mature Themes', 'Violence', ...	103707.0	14351
1	your name.	Movie	1.0	107.0	False	2016.0	2016.0	NaN	Mitsuha and Taki are two total strangers livin...	['CoMix Wave Films']	['Drama', 'Romance', 'Body Swapping', 'Gender ...	[]	58831.0	1453
2	A Silent Voice	Movie	1.0	130.0	False	2016.0	2016.0	NaN	After transferring into a new school, a deaf g...	['Kyoto Animation']	['Drama', 'Shounen', 'Disability', 'Melancholy...	['Bullying', 'Mature Themes', 'Suicide']	45892.0	946
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	TV	10.0	NaN	False	2016.0	2016.0	Fall	Picking up where the second season ended, the ...	['Production I.G']	['Shounen', 'Sports', 'Animeism', 'School Club...	[]	25134.0	2183

Selection of Data

selecting all the rows and columns from 0-4(excluded)

```
In [46]: df.iloc[0:4, 0:4]
```

Out[46]:

	title	mediaType	eps	duration
0	Fullmetal Alchemist: Brotherhood	TV	64.0	NaN
1	your name.	Movie	1.0	107.0
2	A Silent Voice	Movie	1.0	130.0
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	TV	10.0	NaN

Selection of Data

selecting all the rows from 0-10(excluded) and the 0th, 2nd, and 5th columns

```
In [52]: df.iloc[0:10, [0,2,5]]
```

Out[52]:

	title	eps	startYr
0	Fullmetal Alchemist: Brotherhood	64.0	2009.0
1	your name.	1.0	2016.0
2	A Silent Voice	1.0	2016.0
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	10.0	2016.0
4	Attack on Titan 3rd Season: Part II	10.0	2019.0
5	Demon Slayer: Kimetsu no Yaiba	26.0	2019.0
6	Haikyuu!! Second Season	25.0	2015.0
7	Hunter x Hunter (2011)	148.0	2011.0
8	Gintama Kanketsu-hen: Yorozuya yo Eien Nare	1.0	2013.0
9	Gintama (2015)	51.0	2015.0

Filter, Sort & Groupby

- When working with datasets, you will encounter circumstances when you need to sort, filter, or even group your data to make it easier to understand.

Filter

- **df[df[col]** *operator* **number]**
- The **df[df[col]** *operator* **number]** command helps you easily filter out data.

```
In [64]: df[df['watched'] > 1000]
```

Out[64]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	watc
0	Fullmetal Alchemist: Brotherhood	TV	64.0	NaN	False	2009.0	2010.0	Spring	The foundation of alchemy is based on the law ...	['Bones']	['Action', 'Adventure', 'Drama', 'Fantasy', 'M...	['Animal Abuse', 'Mature Themes', 'Violence', ...	103707.0	1
1	your name.	Movie	1.0	107.0	False	2016.0	2016.0	NaN	Mitsuha and Taki are two total strangers livin...	['CoMix Wave Films']	['Drama', 'Romance', 'Body Swapping', 'Gender ...	[]	58831.0	
2	A Silent Voice	Movie	1.0	130.0	False	2016.0	2016.0	NaN	After transferring into a new school, a deaf g...	['Kyoto Animation']	['Drama', 'Shounen', 'Disability', 'Melancholy...	['Bullying', 'Mature Themes', 'Suicide']	45892.0	
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	TV	10.0	NaN	False	2016.0	2016.0	Fall	Picking up where the second season ended, the ...	['Production I.G']	['Shounen', 'Sports', 'Animeism', 'School Club...	[]	25134.0	
4	Attack on Titan 3rd Season: Part II	TV	10.0	NaN	False	2019.0	2019.0	Spring	The battle to retake Wall Maria begins now! Wi...	['Wit Studio']	['Action', 'Fantasy', 'Horror', 'Shounen', 'Da...	['Cannibalism', 'Explicit Violence']	21308.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
11724	Training with Hinako	OVA	1.0	24.0	False	2009.0	2009.0	NaN	If you've ever lacked the enthusiasm for a dai...	['Studio Hibari']	['Ecchi', 'Panty Shots', 'POV', 'Original Work']	[]	3377.0	

Filter

- filtering out with multiple conditions.
- selecting all the records where 'watched' > 1000 and 'eps' = 10

```
In [67]: df[(df['watched'] > 1000) & (df['eps'] == 10)]
```

Out[67]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	wat
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	TV	10.0	NaN	False	2016.0	2016.0	Fall	Picking up where the second season ended, the ...	['Production I.G']	['Shounen', 'Sports', 'Animeism', 'School Club...	[]	25134.0	
4	Attack on Titan 3rd Season: Part II	TV	10.0	NaN	False	2019.0	2019.0	Spring	The battle to retake Wall Maria begins now! Wi...	['Wit Studio']	['Action', 'Fantasy', 'Horror', 'Shounen', 'Da...	['Cannibalism', 'Explicit Violence']	21308.0	
18	Mushishi Zoku Shou 2nd Season	TV	10.0	NaN	False	2014.0	2014.0	Fall	Continuation of Mushishi Zoku Shou.	['Artland']	['Fantasy', 'Seinen', 'Episodic', 'Iyashikei',...	[]	7494.0	
29	Mushishi Zoku Shou	TV	10.0	NaN	False	2014.0	2014.0	Spring	They existed long before anyone can remember. ...	['Artland']	['Fantasy', 'Seinen', 'Episodic', 'Iyashikei',...	[]	8421.0	
112	Hellsing Ultimate	OVA	10.0	49.0	False	2006.0	2012.0	NaN	Alucard is a vampire who works for Hellsing – ...	['MADHOUSE', 'Graphinica', 'Satelight']	['Action', 'Horror', 'Seinen', 'Conspiracy', '...	['Explicit Sex', 'Explicit Violence', 'Mature ...	31968.0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
8193	Piano: The Melody of a Young Girl's Heart	TV	10.0	NaN	False	2002.0	2003.0	Fall	Miu is a young, talented pianist with a passio...	['OLM']	['Slice of Life', 'Classical Music', 'Music', ...	[]	1106.0	

Sort

- **sort_values**
- The **df.sort_values()** command helps in sorting your data in ascending or descending manner.

```
In [68]: df.sort_values('eps')
```

Out[68]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	watching
14577	YOASOBI: Harujion	Music Video	1.0	3.0	False	2020.0	2020.0	NaN	NaN	[]	[]	[]	13.0	0
7243	L'Oeil du Cyclone	Music Video	1.0	5.0	False	2015.0	2015.0	NaN	NaN	[]	['Abstract', 'No Dialogue']	[]	55.0	0
7244	Garo: Vanishing Line Recap	TV Special	1.0	NaN	False	2018.0	2018.0	NaN	NaN	['Mappa']	['Action', 'Sci Fi', 'Conspiracy', 'Recap', 'S...	[]	184.0	9
7245	Mushishi Recap	TV Special	1.0	NaN	False	2006.0	2006.0	NaN	A summary of the first 20 episodes of Mushihi.	[]	['Fantasy', 'Seinen', 'Iyashikei', 'Recap', 'S...	[]	66.0	2
7246	Pokemon Mystery Dungeon: Team Go-Getters Out o...	TV Special	1.0	22.0	False	2006.0	2006.0	NaN	Imagine what it would be like to awaken from s...	['OLM']	['Adventure', 'Fantasy', 'Elemental Powers', '...	[]	6427.0	22
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
14569	Dragon Quest: Dai no Daibouken (2020)	TV	NaN	NaN	False	2020.0	2020.0	Fall	After the defeat of the demon lord Hadlar all ...	['Toei Animation']	['Adventure', 'Fantasy', 'Shounen', 'Demons', ...	[]	0.0	0
14570	Amrita no Kyouen	Movie	NaN	NaN	False	2021.0	2021.0	NaN	The story will center on a high school girl na...	[]	['Horror', 'Insects', 'CG Animation', 'Origina...	[]	0.0	0

Sort

- sorting multiple columns in ascending and descending manner
- sorting the column 'eps' in ascending order and 'duration' in descending order

```
In [72]: df.sort_values(['eps','duration'], ascending = [True,False])
```

Out[72]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	w
12500	Balgwanghaneun Hyeondaesa	Movie	1.0	235.0	False	2014.0	2014.0	NaN	NaN	['Studio Dadashow']	['Drama', 'Korean Animation']	[]	10.0	
441	Ghost in the Shell: Stand Alone Complex - Indi...	Movie	1.0	163.0	False	2006.0	2006.0	NaN	Recently, a number of terrorist attacks have p...	['Production I.G']	['Sci Fi', 'Cyberpunk', 'Cyborgs', 'Hacking', ...	[]	8371.0	
2481	Final Yamato	Movie	1.0	163.0	False	1983.0	1983.0	NaN	The arrival of Aquarius, a watery world that w...	['Office Academy', 'Toei Animation']	['Sci Fi', 'Leijiverse']	[]	263.0	
59	The Disappearance of Haruhi Suzumiya	Movie	1.0	162.0	False	2010.0	2010.0	NaN	Yesterday, Kyon was helping prepare for an SOS...	['Kyoto Animation']	['Drama', 'Sci Fi', 'Shounen', 'Psychological'...	[]	28510.0	
12850	Sangokushi Daisanbu Harukanaru Taichi	Movie	1.0	160.0	False	1994.0	1994.0	NaN	NaN	['Toei Animation']	['Action', 'Ancient China', 'Historical', 'Rom...	[]	12.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
14569	Dragon Quest: Dai no Daibouken (2020)	TV	NaN	NaN	False	2020.0	2020.0	Fall	After the defeat of the demon lord Hadlar all ...	['Toei Animation']	['Adventure', 'Fantasy', 'Shounen', 'Demons', ...	[]	0.0	

Groupby

- The **df.groupby()** command helps split data into separate groups and lets you perform functions on these groups.
-

```
In [87]: df_groupby_eps = df.groupby('eps')
         df_groupby_eps

Out[87]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x000002AD35CDECD0>
```

Groupby

- making use of aggregate functions to compute on grouped data

```
In [102]: df_groupby_eps.mean()
```

Out[102]:

eps	duration	ongoing	startYr	finishYr	watched	watching	wantWatch	dropped	rating	votes
1.0	28.509607	0.001369	2003.866529	2003.846154	1276.087433	16.420145	481.194608	8.882852	2.777137	973.669352
2.0	19.777500	0.035826	2006.616822	2006.484653	1321.290792	62.034268	613.562305	20.219626	2.909734	940.274864
3.0	14.942623	0.048309	2006.665860	2006.475827	1198.423858	69.942029	630.031401	26.644928	2.913252	838.944598
4.0	11.938144	0.051948	2007.035831	2006.996564	1809.842466	80.155844	807.500000	41.493506	2.984646	1271.046931
5.0	10.011765	0.072072	2008.675676	2008.592233	1776.378641	157.459459	802.270270	55.918919	2.825453	1380.000000
...	...	...	...	...	...	...	...	...	...	...
1787.0	11.000000	0.000000	1979.000000	2005.000000	1599.000000	1199.000000	731.000000	1155.000000	3.290000	2342.000000
1818.0	5.000000	0.000000	1994.000000	2013.000000	44.000000	2.000000	22.000000	4.000000	2.166000	26.000000
1888.0	10.000000	1.000000	1993.000000	NaN	NaN	92.000000	135.000000	43.000000	3.582000	64.000000
2367.0	15.000000	1.000000	2005.000000	NaN	NaN	13.000000	11.000000	6.000000	NaN	NaN
2527.0	NaN	1.000000	1969.000000	NaN	NaN	233.000000	216.000000	76.000000	2.737000	139.000000

209 rows × 10 columns

Groupby

- using the agg() function
- we can apply different aggregate functions.
- the below code will display the maximum and minimum rating of animes which are grouped by their votes.
- df.groupby('votes').rating.agg(['max', 'min])

```
In [98]: df.groupby('votes').rating.agg(['max','min'])
```

Out[98]:

votes	max	min
10.0	4.482	1.008
11.0	4.226	1.144
12.0	3.706	1.112
13.0	4.151	1.121
14.0	4.060	1.113
...	...	...
86608.0	4.437	4.437
92416.0	4.184	4.184
97268.0	4.488	4.488
97965.0	4.080	4.080
131067.0	4.501	4.501

3787 rows × 2 columns

Data Cleaning

- If your elders have warned you about how the real world is not what we think it is, how it is messy and uninterpretable, then let me add something more to it.

- Real-world data is just the same: messy and uninterpretable. You first have to clean the data to get the most out of your data and infer meaningful insights. And as I mentioned, Pandas is your best friend, so well, your best friend has got it all covered.

Data Cleaning

- The **df.isnull()** command checks for all the null values in your dataset.

```
In [108]: df.isnull()
```

Out[108]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	watching	wantWatch	d
0	False	False	False	True	False	False	False	False	False	False	False	False	False	False	False	
1	False	False	False	False	False	False	False	True	False	False	False	False	False	False	False	
2	False	False	False	False	False	False	False	True	False	False	False	False	False	False	False	
3	False	False	False	True	False	False	False	False	False	False	False	False	False	False	False	
4	False	False	False	True	False	False	False	False	False	False	False	False	False	False	False	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
14573	False	False	True	True	False	False	False	True	False	False	False	False	False	False	False	
14574	False	False	False	False	False	False	False	True	True	False	False	False	False	False	False	
14575	False	False	False	False	False	False	False	True	True	False	False	False	False	False	False	
14576	False	False	True	True	False	True	True	True	True	False	False	False	False	False	False	
14577	False	False	False	False	False	False	False	True	True	False	False	False	False	False	False	

14578 rows × 18 columns

Data Cleaning

- we can use the sum() function with isnull()
- it will return the sum of null values for each column

```
In [103]: df.isnull().sum()

Out[103]: title             0
          mediaType        68
          eps             359
          duration       5441
          ongoing           0
          startYr         222
          finishYr        444
          sznOfRelease  10811
          description    6405
          studios           0
          tags              0
          contentWarn       0
          watched         222
          watching          0
          wantWatch         0
          dropped           0
          rating         2471
          votes          2459
          dtype: int64
```

Data Cleaning

- The **df.notnull()** command is just the opposite of **df.isnull()** command. This command will check for the non-null values in the dataset.

```
In [109]: df.notnull()
```

Out[109]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	watching	wantWatch	drc
0	True	True	True	False	True	True	True	True	True	True	True	True	True	True	True	
1	True	True	True	True	True	True	True	False	True	True	True	True	True	True	True	
2	True	True	True	True	True	True	True	False	True	True	True	True	True	True	True	
3	True	True	True	False	True	True	True	True	True	True	True	True	True	True	True	
4	True	True	True	False	True	True	True	True	True	True	True	True	True	True	True	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
14573	True	True	False	False	True	True	True	False	True	True	True	True	True	True	True	
14574	True	True	True	True	True	True	True	False	False	True	True	True	True	True	True	
14575	True	True	True	True	True	True	True	False	False	True	True	True	True	True	True	
14576	True	True	False	False	True	False	False	False	False	True	True	True	True	True	True	
14577	True	True	True	True	True	True	True	False	False	True	True	True	True	True	True	

14578 rows × 18 columns

Data Cleaning

- The **dropna()** command drops the rows/columns with missing entries.

```
In [110]: df.dropna()
```

Out[110]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	watchir
149	The Disastrous Life of Saiki K.	TV	120.0	4.0	False	2016.0	2016.0	Summer	Kusuo Saiki is a typical 16-year-old high scho...	['J.C. Staff']	['Comedy', 'Shounen', 'Slice of Life', 'Breaki...	[]	15419.0	269
222	Castlevania Season 2	Web	8.0	25.0	False	2018.0	2018.0	Fall	Trying to save Eastern Europe from extinction ...	['Powerhouse Animation', 'Frederator Studios']	['Action', 'Adventure', 'Drama', 'Horror', '15...	['Explicit Violence', 'Mature Themes', 'Physic...	8010.0	3
236	The Disastrous Life of Saiki K. - Reawakened	Web	6.0	23.0	False	2019.0	2019.0	Winter	Kusuo and his gaggle of self-proclaimed friend...	['J.C. Staff']	['Comedy', 'Shounen', 'Breaking the Fourth Wal...	[]	3136.0	2
238	Katanagatari	TV	12.0	50.0	False	2010.0	2010.0	Winter	In the wake of a rebellion that shook Japan tw...	['WHITE FOX']	['Action', 'Adventure', 'Drama', 'Fantasy', 'R...	[]	14691.0	26
241	Castlevania Season 3	Web	10.0	28.0	False	2020.0	2020.0	Winter	After the fall of Dracula, heroes and villains...	['Powerhouse Animation', 'Frederator Studios']	['Action', 'Adventure', 'Drama', 'Horror', '15...	['Explicit Sex', 'Explicit Violence', 'Mature ...	2975.0	3
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
12033	Forest Fairy Five: Fairy Tale	TV	12.0	14.0	False	2017.0	2017.0	Spring	Second season of Forest Fairy Five.	[]	['Fantasy', 'Chibi', 'Family Friendly', 'CG An...	[]	54.0	

Data Cleaning

- using the "axis" parameter
- "axis = 0" (default) means Row and "axis = 1" means Column
- dropping all the columns with missing entries
- df.dropna(axis = 1)

```
In [114]: df.dropna(axis = 1)
```

Out[114]:

	title	ongoing	studios	tags	contentWarn	watching	wantWatch	dropped
0	Fullmetal Alchemist: Brotherhood	False	['Bones']	['Action', 'Adventure', 'Drama', 'Fantasy', 'M...	['Animal Abuse', 'Mature Themes', 'Violence', ...	14351	25810	2656
1	your name.	False	['CoMix Wave Films']	['Drama', 'Romance', 'Body Swapping', 'Gender ...	[]	1453	21733	124
2	A Silent Voice	False	['Kyoto Animation']	['Drama', 'Shounen', 'Disability', 'Melancholy...	['Bullying', 'Mature Themes', 'Suicide']	946	17148	132
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	False	['Production I.G']	['Shounen', 'Sports', 'Animeism', 'School Club...	[]	2183	8082	167
4	Attack on Titan 3rd Season: Part II	False	['Wit Studio']	['Action', 'Fantasy', 'Horror', 'Shounen', 'Da...	['Cannibalism', 'Explicit Violence']	3217	7864	174
...	...	...	...	...	...	...	...	...
14573	Welcome to Demon School, Iruma-kun 2	False	[]	['Comedy', 'Shounen', 'Demons', 'Monster Schoo...	[]	0	1106	0
14574	Pinocchio-P: Sekai wa Mada Hajimatte Sura Inai	False	[]	['Chibi', 'Vocaloid']	[]	1	7	1
14575	Minagoroshi	False	[]	['Comedy', 'Ecchi', 'No Dialogue', 'Shorts']	[]	0	7	0
14576	Kurayukaba	False	['Makaria']	[]	[]	0	62	0
14577	YOASOBI: Harujion	False	[]	[]	[]	0	6	2

14578 rows × 8 columns

Data Cleaning

- using the "subset" parameter
- it is used for defining in which columns to look for missing values
- dropping all the rows where the "duration" column is NaN
- df.dropna(subset = ['duration'])

```
In [124]: df.dropna(subset = ['duration'])
```

Out[124]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	watching
1	your name.	Movie	1.0	107.0	False	2016.0	2016.0	NaN	Mitsuha and Taki are two total strangers livin...	['CoMix Wave Films']	['Drama', 'Romance', 'Body Swapping', 'Gender ...	[]	58831.0	1453
2	A Silent Voice	Movie	1.0	130.0	False	2016.0	2016.0	NaN	After transferring into a new school, a deaf g...	['Kyoto Animation']	['Drama', 'Shounen', 'Disability', 'Melancholy...	['Bullying', 'Mature Themes', 'Suicide']	45892.0	946
8	Gintama Kanketsu-hen: Yorozuya yo Eien Nare	Movie	1.0	111.0	False	2013.0	2013.0	NaN	While watching a movie, Gintoki comes upon a "...	['Sunrise']	['Action', 'Comedy', 'Drama', 'Sci Fi', 'Shoun...	[]	8454.0	280

Data Cleaning

- **NOTE :** The **dropna()** and **fillna()** commands returns a copy of your object rather than the actual object. To update your object , you need to specify the value of the **inplace** parameter as True.

```
In [136]: df.dropna(inplace = True)
          df.isnull().sum()

Out[136]: title          0
          mediaType      0
          eps            0
          duration       0
          ongoing        0
          startYr        0
          finishYr       0
          sznOfRelease   0
          description    0
          studios        0
          tags           0
          contentWarn    0
          watched        0
          watching       0
          wantWatch      0
          dropped        0
          rating         0
          votes          0
          dtype: int64
```

Data Cleaning

- The **df.fillna()** command does exactly what its name implies: it fills in the missing entries with some value.

```
In [127]: df.fillna(value = "Not Specified")
```

Out[127]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched
0	Fullmetal Alchemist: Brotherhood	TV	64.0	Not Specified	False	2009.0	2010.0	Spring	The foundation of alchemy is based on the law ...	['Bones']	['Action', 'Adventure', 'Drama', 'Fantasy', 'M...	['Animal Abuse', 'Mature Themes', 'Violence', ...	103707.0
1	your name.	Movie	1.0	107.0	False	2016.0	2016.0	Not Specified	Mitsuha and Taki are two total strangers livin...	['CoMix Wave Films']	['Drama', 'Romance', 'Body Swapping', 'Gender ...	[]	58831.0
2	A Silent Voice	Movie	1.0	130.0	False	2016.0	2016.0	Not Specified	After transferring into a new school, a deaf g...	['Kyoto Animation']	['Drama', 'Shounen', 'Disability', 'Melancholy...	['Bullying', 'Mature Themes', 'Suicide']	45892.0
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	TV	10.0	Not Specified	False	2016.0	2016.0	Fall	Picking up where the second season ended, the ...	['Production I.G']	['Shounen', 'Sports', 'Animeism', 'School Club...	[]	25134.0
4	Attack on Titan 3rd Season: Part II	TV	10.0	Not Specified	False	2019.0	2019.0	Spring	The battle to retake Wall Maria begins now! Wi...	['Wit Studio']	['Action', 'Fantasy', 'Horror', 'Shounen', 'Da...	['Cannibalism', 'Explicit Violence']	21308.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
									Second		['Comedy'		

Data Cleaning

- using the “method” parameter
- it specifies how shall we fill the missing values

- method = ‘ffill’/ ‘bfill’
- “ffill” stands for forward fill
- “bfill” stands for backward fill

	Day	Temp
0	Day 1	33.0
1	Day 2	33.0
2	Day 3	35.0
3	Day 4	NaN
4	Day 5	38.0
5	Day 6	37.0
6	Day 7	39.0

Having null values

	Day	Temp
0	Day 1	33.0
1	Day 2	33.0
2	Day 3	35.0
3	Day 4	35.0
4	Day 5	38.0
5	Day 6	37.0
6	Day 7	39.0

ffill

	Day	Temp
0	Day 1	33.0
1	Day 2	33.0
2	Day 3	35.0
3	Day 4	NaN
4	Day 5	38.0
5	Day 6	37.0
6	Day 7	39.0

Having null values

	Day	Temp
0	Day 1	33.0
1	Day 2	33.0
2	Day 3	35.0
3	Day 4	38.0
4	Day 5	38.0
5	Day 6	37.0
6	Day 7	39.0

bfill

Data Cleaning

- using the “method” parameter
- it specifies how shall we fill the missing values

- method = ‘ffill’/ ‘bfill’
- “ffill” stands for forward fill
- “bfill” stands for backward fill

```
In [128]: df.fillna(method = 'ffill')
```

Out[128]:

	title	mediaType	eps	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	watchi
0	Fullmetal Alchemist: Brotherhood	TV	64.0	NaN	False	2009.0	2010.0	Spring	The foundation of alchemy is based on the law ...	['Bones']	['Action', 'Adventure', 'Drama', 'Fantasy', 'M...	['Animal Abuse', 'Mature Themes', 'Violence', ...	103707.0	143
1	your name.	Movie	1.0	107.0	False	2016.0	2016.0	Spring	Mitsuha and Taki are two total strangers livin...	['CoMix Wave Films']	['Drama', 'Romance', 'Body Swapping', 'Gender ...	[]	58831.0	14
2	A Silent Voice	Movie	1.0	130.0	False	2016.0	2016.0	Spring	After transferring into a new school, a deaf g...	['Kyoto Animation']	['Drama', 'Shounen', 'Disability', 'Melancholy...	['Bullying', 'Mature Themes', 'Suicide']	45892.0	9
3	Haikyuu!! Karasuno High School vs Shiratorizaw...	TV	10.0	130.0	False	2016.0	2016.0	Fall	Picking up where the second season ended, the ...	['Production I.G']	['Shounen', 'Sports', 'Animeism', 'School Club...	[]	25134.0	21
4	Attack on Titan 3rd Season: Part II	TV	10.0	130.0	False	2019.0	2019.0	Spring	The battle to retake Wall Maria begins now! Wi...	['Wit Studio']	['Action', 'Fantasy', 'Horror', 'Shounen', 'Da...	['Cannibalism', 'Explicit Violence']	21308.0	32
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
									Second		['Comedy'			

Data Cleaning

- The **df.rename()** command helps us in altering axes labels.

```
In [148]: df.rename(columns = {'eps' : 'Episodes'})
```

Out[148]:

	title	mediaType	Episodes	duration	ongoing	startYr	finishYr	sznOfRelease	description	studios	tags	contentWarn	watched	wat
149	The Disastrous Life of Saiki K.	TV	120.0	4.0	False	2016.0	2016.0	Summer	Kusuo Saiki is a typical 16-year-old high scho...	['J.C. Staff']	['Comedy', 'Shounen', 'Slice of Life', 'Breaki...	[]	15419.0	
222	Castlevania Season 2	Web	8.0	25.0	False	2018.0	2018.0	Fall	Trying to save Eastern Europe from extinction ...	['Powerhouse Animation', 'Frederator Studios']	['Action', 'Adventure', 'Drama', 'Horror', '15...	['Explicit Violence', 'Mature Themes', 'Physic...	8010.0	
236	The Disastrous Life of Saiki K. - Reawakened	Web	6.0	23.0	False	2019.0	2019.0	Winter	Kusuo and his gaggle of self-proclaimed friend...	['J.C. Staff']	['Comedy', 'Shounen', 'Breaking the Fourth Wal...	[]	3136.0	
238	Katanagatari	TV	12.0	50.0	False	2010.0	2010.0	Winter	In the wake of a rebellion that shook Japan tw...	['WHITE FOX']	['Action', 'Adventure', 'Drama', 'Fantasy', 'R...	[]	14691.0	
241	Castlevania Season 3	Web	10.0	28.0	False	2020.0	2020.0	Winter	After the fall of Dracula, heroes and villains...	['Powerhouse Animation', 'Frederator Studios']	['Action', 'Adventure', 'Drama', 'Horror', '15...	['Explicit Sex', 'Explicit Violence', 'Mature ...	2975.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
12033	Forest Fairy Five: Fairy Tale	TV	12.0	14.0	False	2017.0	2017.0	Spring	Second season of Forest Fairy Five.	[]	['Fantasy', 'Chibi', 'Family Friendly', 'CG An...	[]	54.0	

Statistics

- **mean() & median()**
- The **df.mean()** and **df.median()** commands returns the mean and median(respectively) of all columns.

```
In [161]: df.mean()
Out[161]: eps            17.737360
          duration        8.896067
          ongoing         0.000000
          startYr      2013.383427
          finishYr     2013.637640
          watched      1284.127809
          watching      153.179775
          wantWatch     778.689607
          dropped       126.175562
          rating          2.636747
          votes         994.084270
          dtype: float64

In [162]: df.median()
Out[162]: eps            12.0000
          duration        5.0000
          ongoing         0.0000
          startYr      2015.0000
          finishYr     2015.0000
          watched       425.0000
          watching       52.0000
          wantWatch     399.0000
          dropped        50.0000
          rating          2.5925
          votes         345.5000
          dtype: float64
```

Statistics

- The **df.corr()** command returns the correlation between columns.
-

```
In [163]: df.corr()
```

Out[163]:

	eps	duration	ongoing	startYr	finishYr	watched	watching	wantWatch	dropped	rating	votes
eps	1.000000	-0.074737	NaN	-0.122137	-0.063983	-0.006473	0.057037	-0.009882	0.028430	0.028331	0.000412
duration	-0.074737	1.000000	NaN	0.006958	0.000754	0.165776	0.141770	0.229727	0.096362	0.314129	0.160806
ongoing	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
startYr	-0.122137	0.006958	NaN	1.000000	0.992003	-0.084581	0.052064	-0.062616	-0.092981	-0.036820	-0.070998
finishYr	-0.063983	0.000754	NaN	0.992003	1.000000	-0.094167	0.049839	-0.071744	-0.098449	-0.033899	-0.079358
watched	-0.006473	0.165776	NaN	-0.084581	-0.094167	1.000000	0.841623	0.891287	0.749934	0.460430	0.993779
watching	0.057037	0.141770	NaN	0.052064	0.049839	0.841623	1.000000	0.854491	0.783481	0.405857	0.873878
wantWatch	-0.009882	0.229727	NaN	-0.062616	-0.071744	0.891287	0.854491	1.000000	0.723434	0.459878	0.901827
dropped	0.028430	0.096362	NaN	-0.092981	-0.098449	0.749934	0.783481	0.723434	1.000000	0.176824	0.790161
rating	0.028331	0.314129	NaN	-0.036820	-0.033899	0.460430	0.405857	0.459878	0.176824	1.000000	0.449626
votes	0.000412	0.160806	NaN	-0.070998	-0.079358	0.993779	0.873878	0.901827	0.790161	0.449626	1.000000

Statistics

- The **df.std()** command returns the standard deviation of all the columns.
-

```
In [164]: df.std()

Out[164]: eps            33.543303
          duration        9.946291
          ongoing         0.000000
          startYr         5.189832
          finishYr        5.096331
          watched      2440.286697
          watching      311.619979
          wantWatch    1146.541640
          dropped       219.940898
          rating          0.751512
          votes        1897.492886
          dtype: float64
```

Statistics

- The **df.max()** and **df.min()** commands returns the highest and lowest value in each columns.
-

```
In [165]: df.max()

Out[165]: title                                          gdgd men's party
          mediaType                                                   Web
          eps                                                       744.0
          duration                                                  120.0
          ongoing                                                   False
          startYr                                                  2020.0
          finishYr                                                 2020.0
          sznOfRelease                                             Winter
          description     ”Wereman" Shushu's life is turned upside down ...
          studios                                                       []
          tags            ['Sports', 'Basketball', 'Disability', 'Runnin...
          contentWarn                                                   []
          watched                                                  22138.0
          watching                                                    3461
          wantWatch                                                  12044
          dropped                                                     2442
          rating                                                     4.453
          votes                                                    16938.0
          dtype: object
```

Combining Data Frames

- Combining datasets is necessary when you have multiple datasets yet want to study all their data simultaneously.

Combining Data Frames

- The **pd.concat()** command lets you combine data across rows or columns.
-

```
df2 = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
                    'B': ['B0', 'B1', 'B2', 'B3'],
                    'C': ['C0', 'C1', 'C2', 'C3'],
                    'D': ['D0', 'D1', 'D2', 'D3']},
                    index=[0, 1, 2, 3])
df3 = pd.DataFrame({'A': ['A4', 'A5', 'A6', 'A7'],
                    'B': ['B4', 'B5', 'B6', 'B7'],
                    'C': ['C4', 'C5', 'C6', 'C7'],
                    'D': ['D4', 'D5', 'D6', 'D7']},
                     index=[0, 1, 2, 3])
df4 = pd.DataFrame({'A': ['A8', 'A9', 'A10', 'A11'],
                    'B': ['B8', 'B9', 'B10', 'B11'],
                    'C': ['C8', 'C9', 'C10', 'C11'],
                    'D': ['D8', 'D9', 'D10', 'D11']},
                    index=[8,9,10,11])
```

Combining Data Frames

- The **pd.concat()** command lets you combine data across rows or columns.

```
In [205]: pd.concat([df2,df3,df4]) # by default, concatenation happens row-wise
```

Out[205]:

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
0	A4	B4	C4	D4
1	A5	B5	C5	D5
2	A6	B6	C6	D6
3	A7	B7	C7	D7
8	A8	B8	C8	D8
9	A9	B9	C9	D9
10	A10	B10	C10	D10
11	A11	B11	C11	D11

Any Questions ?

Thank You

Reference

https://www.analyticsvidhya.com/blog/2022/08/the-ultimate-guide-to-pandas-for-data-science/?utm_source=reading_list??